Republic of Tunisia
Ministry of Higher Education and Scientific Research
Esprit School of Engineering

FoodRescue TN

Food-waste reduction: from surplus to someone's table

Technical Design Report

Course: Distributed Web Application

Team members and microservices

Student	Microservice
Malek Hammami	Donation Offer Service
Med Sherif	Matching & Pickup Service
Salma Boubakri	Partner Registry & Trust Service
Yasmine Ben Abdelkader	Impact Analytics Service
Rayen Ghileb	Volunteer Driver Service
Mohamed Fadhel	Surprise Basket Marketplace Service
Shared work: User & Auth + Notification, API Gateway, Eureka, Config Server, RabbitMQ (whole team)	

Supervisor: [Badiaa Boudid]

Academic year: 2026

Abstract

Every day, large quantities of edible food from bakeries, restaurants, hotels and supermarkets are thrown away, while families and charities in Tunisia lack food and landfilled waste emits methane. **FoodRescue TN** is a distributed web platform that routes this surplus to those who need it: donors publish surplus offers, the system matches each offer with the nearest compatible association, schedules the pickup with a volunteer driver, and measures the resulting impact in kilograms of food saved, meals provided and CO_2e avoided. Citizens can also buy discounted "surprise baskets" made from unsold stock. The solution follows a microservice architecture of six business services (Donation Offer, Matching & Pickup, Partner Registry & Trust, Impact Analytics, Volunteer Driver, Surprise Basket Marketplace), each with its own database (MongoDB, PostgreSQL/PostGIS, Redis) and built with Spring Boot, supported by an API Gateway, Eureka, a Config Server, Keycloak and RabbitMQ. Services communicate synchronously through REST/OpenFeign for validations and commands, and asynchronously through RabbitMQ events for facts and notifications. The project is managed with the Scrum agile methodology, organised in short sprints with a contract-first approach to APIs and events. This report presents the topic, the project methodology, the actors and features, the breakdown into microservices, the data of each service, their interactions and the overall architecture.

Keywords: food waste, microservices, Spring Boot, RabbitMQ, Keycloak, geo-matching, event-driven architecture, Scrum.

Contents

Abstract i

List of Figures v

List of Tables vi

1 Introduction and Project Description 1
- 1.1 Context . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Proposed Solution . . . 2
- 1.4 Objectives . . . 2

2 Project Management Methodology 3
- 2.1 Introduction . . . 3
- 2.2 Agile Methodology . . . 3
- 2.3 The Scrum Framework . . . 3
  - 2.3.1 General presentation . . . 3
  - 2.3.2 Why Scrum for FoodRescue TN . . . 3
  - 2.3.3 Scrum roles . . . 4
  - 2.3.4 Scrum artifacts . . . 4
  - 2.3.5 Scrum events . . . 4
- 2.4 Conclusion . . . 4

3 Requirements Specification and Project Planning 6
- 3.1 Introduction . . . 6
- 3.2 Requirement Analysis . . . 6
  - 3.2.1 Actors Identification . . . 6
  - 3.2.2 Global Use Case Diagram . . . 6
  - 3.2.3 Functional Requirements . . . 6
  - 3.2.4 Non-Functional Requirements . . . 9
- 3.3 Project Planning . . . 10
  - 3.3.1 Product Backlog . . . 10
  - 3.3.2 Identification of Scrum Roles . . . 12
  - 3.3.3 Sprint Planning . . . 12
- 3.4 Work Environment . . . 13
  - 3.4.1 Software Development Environment . . . 13
- 3.5 Conclusion . . . 13

4 Microservice Architecture **17**
4.1 Why a Microservice Architecture . . . 17
4.2 Decomposition Principle . . . 18
4.3 Service Catalogue . . . 19
4.3.1 Donation Offer Service . . . 19
4.3.2 Matching & Pickup Service . . . 20
4.3.3 Partner Registry & Trust Service . . . 20
4.3.4 Impact Analytics Service . . . 21
4.3.5 Volunteer Driver Service . . . 21
4.3.6 Surprise Basket Marketplace Service . . . 22
4.3.7 Choice between donation and surprise basket . . . 22
4.4 Infrastructure Services . . . 22
4.5 Technology Choices . . . 22
4.6 Main Risks and Mitigation . . . 23
4.7 Chapter Conclusion . . . 23

5 Data Managed by Each Service **26**
5.1 Donation Offer Service (MongoDB) . . . 26
5.2 Matching & Pickup Service (PostgreSQL + PostGIS) . . . 28
5.3 Partner Registry & Trust Service (PostgreSQL) . . . 30
5.4 Impact Analytics Service (PostgreSQL) . . . 31
5.5 Volunteer Driver Service (PostgreSQL) . . . 32
5.6 Surprise Basket Marketplace Service (PostgreSQL) . . . 34
5.7 User & Auth and Notification (Keycloak / PostgreSQL) . . . 35
5.7.1 Identity data in Keycloak . . . 35
5.7.2 Application data in PostgreSQL . . . 36
5.8 Data Ownership Summary . . . 37

6 Interactions between Services **40**
6.1 Communication Principles . . . 40
6.2 Service Dependencies . . . 42
6.3 Synchronous Interactions (REST / OpenFeign) . . . 42
6.3.1 Main synchronous interactions . . . 44
6.3.2 Reservation of stock . . . 44
6.3.3 Response codes . . . 44
6.3.4 Example of OpenFeign communication . . . 45
6.4 Asynchronous Interactions (RabbitMQ Events) . . . 46
6.4.1 RabbitMQ communication model . . . 46
6.4.2 Domain events in FoodRescue TN . . . 46
6.4.3 Advantages of asynchronous communication . . . 47
6.5 Sequence Diagrams . . . 48
6.5.1 Main flow: from offer to impact . . . 48
6.5.2 Surprise basket flow . . . 49

6.6 Reliability Aspects . . . 51
6.6.1 Timeouts and error handling . . . 51
6.6.2 Message delivery and retries . . . 53
6.6.3 Idempotent event consumption . . . 53
6.6.4 Data consistency and concurrent operations . . . 54
6.6.5 Reliability measures by service . . . 54
6.7 Chapter Conclusion . . . 55

7 Overall Architecture 57
7.1 Global Microservices Architecture . . . 57
7.2 Microservices and Their Responsibilities . . . 58
7.3 Explanation of the Request Path . . . 60
7.4 Example: Publishing and Processing a Donation Offer . . . 62
7.5 Deployment View . . . 63

8 Other Design Elements 68
8.1 Security (Keycloak, JWT, Roles) . . . 68
8.1.1 Authentication with Keycloak . . . 68
8.1.2 JSON Web Token (JWT) . . . 69
8.1.3 Role-Based Access Control (RBAC) . . . 69
8.1.4 Authorization and Access Control . . . 70
8.1.5 Security Objectives . . . 70
8.2 Business Rules and Assumptions . . . 70
8.2.1 User and Partner Management . . . 71
8.2.2 Donation Offer Management . . . 71
8.2.3 Matching and Pickup Management . . . 71
8.2.4 Surprise Basket Marketplace . . . 72
8.2.5 Ratings and Impact Calculation . . . 72
8.2.6 Main Business Assumptions . . . 73
8.3 Team Organization . . . 73
8.3.1 Development Responsibilities . . . 73
8.3.2 Work Organization . . . 74
8.3.3 Collaboration and Coordination . . . 74
8.4 Technology Stack Summary . . . 75
8.4.1 Database Allocation . . . 75
8.4.2 Technology Selection . . . 75

General Conclusion 79
Conclusion . . . 80

List of Figures

2.1 The Scrum cycle applied to FoodRescue TN 5

3.1 Enter Caption 7
3.2 Global use case diagram of FoodRescue TN 8
3.3 Gantt chart of the project (W1 to W7: weeks from the project start) . . . 16

4.1 Decomposition of FoodRescue TN into bounded contexts 19

5.1 Class diagram of the Donation Offer service 27
5.2 Class diagrams of the Matching & Pickup service (left) and of the Impact Analytics service (right) 29
5.3 Class diagram of the Partner Registry & Trust service 30
5.4 Class diagrams of the Volunteer Driver service (top) and of the Surprise Basket Marketplace service (bottom) 33
5.5 Class diagram of the common User, Auth & Notification service 36

6.1 Enter Caption 42
6.2 Overall service dependencies of FoodRescue TN 43
6.3 General asynchronous communication through RabbitMQ 46
6.4 Main sequence: from offer publication to impact calculation 50
6.5 Surprise basket sequence: from order to impact calculation 52

7.1 Overall microservices architecture of FoodRescue TN 67

List of Tables

2.1 The three Scrum roles . . . 4
2.2 Scrum events . . . 5

3.1 Actors of FoodRescue TN . . . 7
3.3 Functional requirements . . . 8
3.2 Use cases of the global diagram and related requirements . . . 10
3.5 Product backlog . . . 10
3.4 Non-functional requirements . . . 12
3.6 Scrum roles in the FoodRescue TN team . . . 13
3.7 Sprint plan . . . 14
3.8 Work of each service owner per sprint . . . 15

4.1 Catalogue of the business microservices . . . 20
4.2 Infrastructure and shared services . . . 23
4.3 Technology choices and justification . . . 24
4.4 Main risks and mitigation measures . . . 25

5.1 Classes of the Donation Offer service . . . 27
5.2 Tables of the Matching & Pickup service . . . 29
5.3 Tables of the Partner Registry & Trust service . . . 31
5.4 Tables of the Impact Analytics service . . . 32
5.5 Tables of the Volunteer Driver service . . . 34
5.6 Tables of the Surprise Basket Marketplace service . . . 35
5.7 Tables of the User & Auth and Notification service . . . 36
5.8 Data ownership by service . . . 38
5.9 References between services (identifiers without foreign keys) . . . 39

6.1 Comparison of communication mechanisms . . . 41
6.2 Dependencies between services . . . 44
6.3 REST endpoints used in inter-service communication . . . 47
6.4 Event contracts of FoodRescue TN . . . 49
6.5 Behaviour of the caller when a called service is unavailable . . . 55

7.1 Main business events exchanged through RabbitMQ . . . 62
7.2 Microservices and data storage technologies . . . 64

8.1 Main user roles and permissions . . . 77
8.2 Technology stack of FoodRescue TN . . . 78

Chapter 1

Introduction and Project Description

1.1 Context

Food is lost or wasted at every stage of the supply chain, from production to the final consumer. In Tunisia, as in many countries of the Mediterranean region, bakeries, restaurants, hotels, supermarkets and event caterers throw away every day edible food that is still safe to eat: unsold bread, prepared meals that were not served, fresh produce and packaged products close to their expiry date. Most of this surplus ends up in landfills, where the decomposition of organic matter emits methane, a greenhouse gas far more potent than carbon dioxide over the short term. Wasting food also wastes the water, energy and labour used to produce it.

At the same time, many families face food insecurity, and a large network of associations and charities works to feed them with limited resources, limited transport and little visibility on where surplus is available. Donors are generally willing to give, but they lack a simple and fast channel to do so before the food spoils.

1.2 Problem Statement

The surplus of donors and the needs of associations exist in the same cities, but nothing connects them in time. The problem can be summarised in four points.

- **No channel for donors:** a bakery or a hotel that wants to give its surplus has no simple way to find a charity, and the pickup window is short (often a few hours).
- **No coordination for associations:** associations cannot know which surplus is available nearby, nor organise the transport with limited volunteers.
- **No trust or traceability:** the quality of the food, the reliability of the partners and the path of each donation are not recorded.
- **No measure of the impact:** nobody can tell how many kilograms of food, how many meals and how much CO_2e were actually saved.

The central question of the project is therefore: *how can a surplus of edible food be routed, quickly and in a traceable way, from the donor who has it to the association or citizen who needs it, and how can the result be measured?*

1.3 Proposed Solution

FoodRescue TN is a distributed web platform that follows the surplus from the donor to someone's table. A donor publishes a surplus offer (type of food, quantity, pickup window, photo). The platform matches the offer with the nearest compatible association, schedules the pickup with an available volunteer driver, and records the delivery. Food that is not suited to donation can instead be sold at a reduced price to citizens as "surprise baskets". Every delivery and every collected basket feeds an impact dashboard showing the kilograms of food saved, the meals provided and the CO_2e avoided. Partners are verified, and the quality of each exchange is tracked through food-safety checklists and ratings.

The platform is built as a microservice architecture of six business services, each owned by one team member and holding its own database, supported by shared infrastructure services (API Gateway, Eureka, Config Server, Keycloak, RabbitMQ) and a front-end application consuming the APIs exposed through the gateway.

1.4 Objectives

The project pursues the following objectives.

- **Reduce food waste** by giving donors a fast and simple way to publish their surplus.
- **Route food quickly** to the most suitable association through geographic matching and volunteer drivers.
- **Build trust and traceability** through partner verification, food-safety checklists, ratings and status history.
- **Measure the impact** in kilograms saved, meals provided and CO_2e avoided, per governorate and per donor.
- **Apply the course concepts** of distributed web applications: microservices, API Gateway, service discovery, centralised configuration, secured access, synchronous and asynchronous communication, and containerisation.

Chapter 2

Project Management Methodology

2.1 Introduction

The way a project is organised has a direct effect on its success, especially for a distributed application developed in parallel by six people under a fixed deadline. This chapter presents the methodology chosen by the team: the Agile approach, and within it the Scrum framework.

2.2 Agile Methodology

Agile is a family of iterative and incremental approaches to software development. Instead of specifying everything in advance and delivering at the end, as in a sequential (waterfall) model, an agile team delivers the product in short cycles, collects feedback after each cycle and adapts its plan. The *Agile Manifesto* summarises this philosophy through four values:

- individuals and interactions over processes and tools;
- working software over comprehensive documentation;
- customer collaboration over contract negotiation;
- responding to change over following a plan.

2.3 The Scrum Framework

2.3.1 General presentation

Scrum is the most widely used agile framework. It organises the work in fixed-length iterations called *sprints*, at the end of which the team delivers a potentially usable product increment. A Scrum team is small, self-organised and cross-functional, and it relies on three pillars: transparency of the work, regular inspection of the product and of the process, and adaptation when something deviates.

2.3.2 Why Scrum for FoodRescue TN

Scrum fits this project for the following reasons.

- **Changing requirements:** matching rules, event payloads and the choice between donation and surprise basket will evolve during the project.
- **Incremental services:** each microservice starts minimal (CRUD) and grows step by step (communication, events, security).
- **Early integration:** each sprint review forces the independently developed services to work together.
- **Visible progress:** the sprint board and the reviews make the progress of every service visible to the whole team and to the Product Owner.

2.3.3 Scrum roles

Scrum defines three roles, summarised in Table 2.1. Their assignment to our team is given in Chapter 3.

Table 2.1: The three Scrum roles

Role	Responsibility
Product Owner	Represents the stakeholders, owns the product backlog, defines priorities and acceptance criteria, and validates the increments.
Scrum Master	Ensures that Scrum is understood and applied, facilitates the events and removes the obstacles blocking the team.
Development Team	Designs, implements, tests and documents the product; it is self-organised and jointly responsible for the increment.

2.3.4 Scrum artifacts

- **Product backlog:** the ordered list of everything the product needs, written as user stories and prioritised by the Product Owner.
- **Sprint backlog:** the items selected for the current sprint, with the tasks needed to deliver them.
- **Increment:** the working result of a sprint, which meets the *Definition of Done*.

2.3.5 Scrum events

Table 2.2 lists the events of a sprint and Figure 2.1 shows the resulting cycle.

2.4 Conclusion

Scrum gives the team a light but disciplined way to work: short sprints, a prioritised backlog and regular reviews. The next chapter applies it to FoodRescue TN by specifying the requirements, building the product backlog and planning the sprints.

Table 2.2: Scrum events

Event	**Frequency**	**Purpose**
Sprint	One to two weeks	Container of all the other events; produces an increment.
Sprint planning	Start of each sprint	Select the backlog items and define the sprint goal.
Daily stand-up	Short (15 minutes), two to three times a week in our case	Share what was done, what is next and what blocks.
Sprint review	End of each sprint	Demonstrate the increment and collect feedback.
Sprint retrospective	After the review	Identify what went well and what to improve.

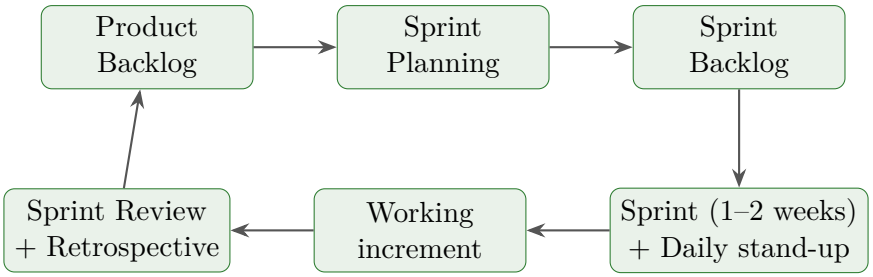


Figure 2.1: The Scrum cycle applied to FoodRescue TN

Chapter 3

Requirements Specification and Project Planning

3.1 Introduction

This chapter specifies what FoodRescue TN must do and how the work is organised. It first analyses the requirements (actors, global use case diagram, functional and non-functional requirements). It then presents the project planning following Scrum (product backlog, Scrum roles, sprint planning and Gantt chart). It ends with the work environment used by the team.

3.2 Requirement Analysis

3.2.1 Actors Identification

An actor is a person or a system that interacts with the platform. Table 3.1 lists the five actors of FoodRescue TN.

3.2.2 Global Use Case Diagram

Figure 3.2 shows the global use case diagram of FoodRescue TN. It groups the use cases of the five actors into five functional domains: donation management, charity matching, volunteer pickups, discounted baskets, and administration and impact. Registration and login through Keycloak are available to every actor and are not drawn. The food-safety checklist and the ratings (FR13 and FR14), done by the association and the donor after a delivery, are also left out of the figure to keep it readable. Table 3.2 links each use case of the diagram to the functional requirements of Table 3.3.

3.2.3 Functional Requirements

Table 3.3 lists the functional requirements, the actor concerned and the service responsible. They are the source of the product backlog of Section 3.3.1.

Table 3.1: Actors of FoodRescue TN

Actor	**Description**	**Main goals**
Donor	Bakery, restaurant, hotel or shop that has edible surplus.	Publish surplus quickly, avoid waste, sell unsold stock as baskets and validate the pickup code of the citizens, see the impact of its donations.
Association / charity	Verified organisation that distributes food to people in need.	Declare its needs, receive compatible surplus nearby, accept or reject matches, follow pickups, check food safety, rate donors.
Volunteer driver	Volunteer who transports food from donors to associations.	Declare availability, be assigned to nearby pickups, view the assigned pickups and update their status.
Citizen	Person who buys discounted surprise baskets.	Save money on good food, reduce waste, reserve a basket and collect it with a pickup code.
Admin	Administrator of the platform.	Verify donors and charities, follow the reported issues, monitor the impact.

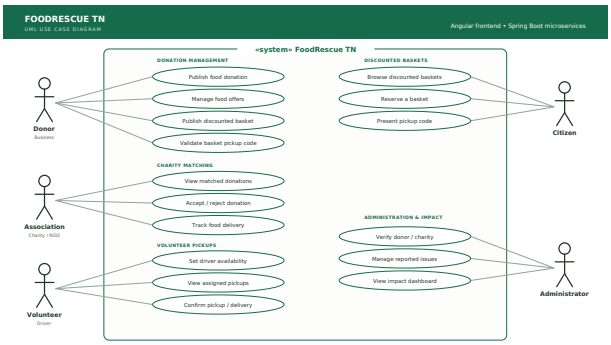


Figure 3.1: Enter Caption

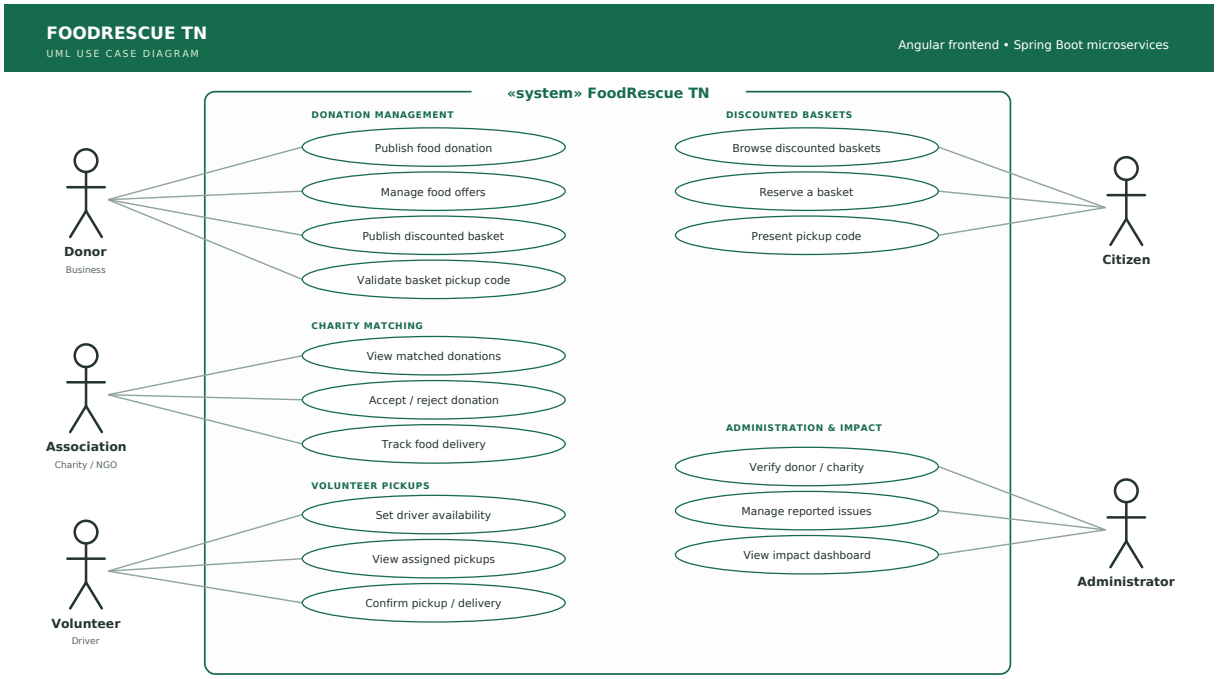


Figure 3.2: Global use case diagram of FoodRescue TN

Table 3.3: Functional requirements

ID	Requirement	Actor	Service
FR1	Register and log in with a role (donor, association, driver, citizen, admin)	All	Common User + Notification (Keycloak)
FR2	Create and edit a partner profile with one or more branches (address, governorate, location, capacity)	Donor, Association	Partner Registry & Trust
FR3	Verify or reject a partner and review the issues reported about it (failed safety checks, negative reviews)	Admin	Partner Registry & Trust
FR4	Publish a surplus offer (food items, quantity, pickup window, photo, location)	Donor	Donation Offer
FR5	Edit or cancel an offer that has no active reservation; automatic expiry at the end of the pickup window	Donor	Donation Offer
FR6	Browse offers by category and status	Association, Admin	Donation Offer
FR7	Match an offer, when it is published, with the nearest compatible association according to the needs it has declared (category, quantity, priority)	Association (automatic)	Matching & Pickup

ID	Requirement	Actor	Service
FR8	Confirm or reject a proposed match and list the matched donations	Association	Matching & Pickup
FR9	Schedule a pickup and track its status until delivery	Association, Driver	Matching & Pickup
FR10	Declare availability slots and vehicle, and set the maximum travel distance	Driver	Volunteer Driver
FR11	Find an available driver near the pickup point, assign the driver and book the slot	Driver (automatic)	Volunteer Driver, Matching & Pickup
FR12	Update the status of a pickup (in transit, delivered)	Driver	Matching & Pickup
FR13	Fill in a food-safety checklist when receiving a delivery	Association	Partner Registry & Trust
FR14	Rate and review a partner or a driver after an exchange	Association, Donor	Partner Registry & Trust, Volunteer Driver
FR15	Publish a surprise basket built from the unsold stock of an offer	Donor	Surprise Basket Marketplace, Donation Offer
FR16	Browse baskets, reserve one (stock reserved in the Donation Offer service, simulated payment) and get a unique pickup code	Citizen	Surprise Basket Marketplace, Donation Offer
FR17	Collect a basket: the citizen presents the pickup code and the donor validates it	Citizen, Donor	Surprise Basket Marketplace
FR18	Compute kg saved, meals provided and CO_2e avoided from the `DeliveryCompleted` and `BasketCollected` events	System	Impact Analytics
FR19	Display the impact dashboard per governorate and per donor, and the leaderboard	Admin, Donor	Impact Analytics
FR20	Produce a monthly impact report	Admin	Impact Analytics
FR21	Send notifications (e-mail or push) on key events, for example a confirmed match	All	Common User + Notification
FR22	(Optional) Forecast the demand for surplus	Admin	Impact Analytics (FastAPI)

3.2.4 Non-Functional Requirements

Table 3.4 presents the quality requirements of the platform.

Table 3.2: Use cases of the global diagram and related requirements

Domain	Use case	Actor	Requirements
Donation management	Publish food donation	Donor	FR4
	Manage food offers	Donor	FR5
Charity matching	View matched donations	Association	FR7, FR8
	Accept / reject donation	Association	FR8
	Track food delivery	Association	FR9
Volunteer pickups	Set driver availability	Volunteer driver	FR10
	View assigned pickups	Volunteer driver	FR11
	Confirm pickup / delivery	Volunteer driver	FR12
Discounted baskets	Publish discounted basket	Donor	FR15
	Validate basket pickup code	Donor	FR17
	Browse discounted baskets	Citizen	FR16
	Reserve a basket	Citizen	FR16
	Present pickup code	Citizen	FR17
Administration and impact	Verify donor / charity	Admin	FR3
	Manage reported issues	Admin	FR3
	View impact dashboard	Admin	FR19, FR20

3.3 Project Planning

3.3.1 Product Backlog

Table 3.5: Product backlog

ID	Service	User story	Priority	Sprint
US-01	Donation Offer	As a donor, I want to publish a surplus offer (food items, quantity, pickup window, photo) so that an association can collect it.	Must	1
US-02	Donation Offer	As a donor, I want to edit or cancel an offer that has no active reservation so that I can correct my mistakes.	Should	1
US-05	Partner Registry	As a donor or an association, I want to create my profile and have it verified by an admin so that partners can trust me.	Must	1
US-07	Volunteer Driver	As a volunteer driver, I want to declare my availability slots so that I can be assigned to pickups.	Must	1

ID	Service	User story	Priority	Sprint
US-09	Surprise Basket	As a donor, I want to publish a surprise basket from the unsold stock of an offer so that I can sell it at a reduced price.	Should	1
US-03	Matching & Pickup	As an association, I want to declare my needs and be matched with the nearest compatible offer so that food is collected quickly.	Must	2
US-08	Matching & Pickup, Volunteer Driver	As a volunteer driver, I want to be assigned to a nearby pickup, view it and update its status so that deliveries are tracked.	Must	2
US-10	Surprise Basket	As a citizen, I want to reserve a discounted basket so that the stock is kept for me and I get a unique pickup code.	Should	2
US-15	Common User + Notification	As a user, I want to register and log in with my role so that I only access what concerns me.	Must	2
US-04	Matching & Pickup	As an association, I want to accept or reject a match and follow the pickup status so that I know when the food arrives.	Must	3
US-16	Common User + Notification	As an association or a driver, I want to be notified when a match is confirmed so that I can act immediately.	Should	3
US-06	Partner Registry	As an association, I want to fill in a food-safety checklist and rate the donor so that trust is built.	Should	4
US-11	Surprise Basket	As a citizen, I want to present my pickup code and as a donor, I want to validate it, so that the basket is handed over and the sale is completed.	Should	4
US-12	Impact Analytics	As an admin, I want a dashboard of kg saved, meals and CO_2e avoided so that the impact is measurable.	Should	4
US-13	Impact Analytics	As an admin, I want a monthly report and a leaderboard so that I can communicate the results.	Could	4
US-14	Impact Analytics	As an admin, I want a demand forecast so that associations can anticipate surplus.	Could	5

3.3.2 Identification of Scrum Roles

Table 3.6 assigns the Scrum roles in our team.

Table 3.4: Non-functional requirements

Requirement	Description
Security	Authentication with Keycloak (OAuth2 / OpenID Connect), JWT tokens checked at the API Gateway, role-based access to every endpoint, no password handled by the business services.
Scalability	Each service can be scaled independently; clients find instances through Eureka.
Availability and fault tolerance	The failure of one service (for example Impact Analytics) must not block the main flow; resilience mechanisms (timeouts, retries, circuit breaker with Resilience4j) protect synchronous calls, and a service that cannot be reached answers with a 503 error.
Loose coupling	Services do not share databases and reference each other by identifier only. Immediate reads and stock reservations use REST calls (OpenFeign), and facts are exchanged as domain events through RabbitMQ.
Data consistency	The Donation Offer service is the only owner of the stock: reservations are made synchronously, and released when a workflow is aborted. Event consumers ignore duplicate `eventId` values (idempotent processing).
Performance	Matching and dashboard queries must stay responsive; geospatial indexes and pre-computed aggregates are used for this purpose.
Traceability	Status changes of pickups and orders are recorded with their date and author.
Usability	A responsive front-end, with a simple publication flow for donors.

3.3.3 Sprint Planning

The project is planned as a seven-week delivery split into six sprints: a two-week inception sprint (Sprint 0) followed by five one-week development sprints (Sprints 1 to 5). Sprint 0 is dedicated to the design: no business code is written before the design is reviewed by the Product Owner. The short development sprints suit the limited delivery time, since they give an integrated increment to review every week. Each sprint starts with a planning meeting and ends with a review and a retrospective. Table 3.7 gives the goal and the main work of each sprint, and Table 3.8 details the work of each service owner.

Gantt chart. Figure 3.3 shows the schedule of the sprints and of the main tasks over the seven weeks of the project. The front-end and the documentation run in parallel with the development of the services.

Table 3.6: Scrum roles in the FoodRescue TN team

Role	Assigned to	Remark
Product Owner	charity donor	Validates each increment and sets the priorities with the team.
Scrum Master	Mrs Badiaa bouhdid	Facilitates the events and removes obstacles.
Development Team	Malek Hammami, Med Sherif, Salma Boubakri, Yasmine Ben Abdelkader, Rayen Ghileb, Mohamed Fadhel	Each member owns one business microservice; the infrastructure and the common User + Notification service are shared by all.

3.4 Work Environment

3.4.1 Software Development Environment

Table **??** lists the technologies and tools used, grouped by purpose. They follow the mandatory stack of the module.

3.5 Conclusion

This chapter specified the requirements of FoodRescue TN: five actors, a global use case diagram grouped in five functional domains, twenty-two functional requirements and the main quality requirements. It then organised the work with Scrum: a product backlog of user stories prioritised with MoSCoW, the assignment of the Scrum roles, six sprints (a two-week inception sprint and five one-week development sprints) with the work of each service owner, and a Gantt chart. Finally, it presented the software environment of the team. The next chapter explains how the platform is decomposed into microservices.

Table 3.7: Sprint plan

Sprint	**Weeks**	**Goal**	**Main work and increment**
Sprint 0	W1 – W2	Inception and design	Topic, actors and features; breakdown into microservices; data model, REST and event contracts; architecture diagram; Git repository, project board and microservice projects. *Increment:* validated design and technical document.
Sprint 1	W3	Foundations and core services	Config Server, Eureka and API Gateway; Docker Compose for the infrastructure (databases, RabbitMQ, Keycloak); CI pipeline; CRUD, database and Swagger documentation of each service; front-end skeleton. *Increment:* all services start, register in Eureka and are reachable through the gateway.
Sprint 2	W4	Security and business flow	Keycloak realm, roles and JWT checked at the gateway; synchronous calls with OpenFeign and Resilience4j; stock reservation in the Donation Offer service; nearest-association matching and driver assignment; basket orders with pickup codes; front-end screens for donors and associations. *Increment:* a donation can be published, matched and assigned to a driver.
Sprint 3	W5	Events and notifications	RabbitMQ exchanges, queues and events (`OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`); idempotent consumers; Notification service; front-end screens for drivers and citizens. *Increment:* the donation flow runs end to end with events and notifications.
Sprint 4	W6	Trust and impact	Ratings and food-safety checklist; basket collection and the `BasketCollected` event; impact aggregation, dashboard, monthly report and leaderboard; front-end dashboard and basket screens. *Increment:* the complete functional scope.
Sprint 5	W7	Hardening and release	Failure scenarios (unavailable service, expired offer, released reservation); integration and end-to-end tests; optional demand forecast; Dockerization of the whole stack; documentation and demo rehearsal. *Increment:* final release.

Table 3.8: Work of each service owner per sprint

Service (owner)	**Sprint 1**	**Sprint 2**	**Sprint 3**	**Sprint 4**	**Sprint 5**
Donation Offer (M. Hammami)	Offer CRUD, MongoDB indexes, Swagger	Donor check in Partner Registry; stock reservation and release endpoints for Matching and Basket	Publish `OfferPublished`; expiry task for offers and unused reservations	Cancel rules; offer status in the front-end	Integration tests, API documentation
Matching & Pickup (M. Sherif)	Match, need and pickup model, PostGIS setup	Association needs; nearest-association query; calls to Offer, Partner and Driver; driver assignment	Consume `OfferPublished`; publish `MatchConfirmed` and `DeliveryCompleted`; status history	Pickup status transitions and tracking endpoints	Failure scenarios (no driver, expired offer); tests
Partner Registry & Trust (S. Boubakri)	Partner profiles, branches and verification workflow	Donor-check, branch capacity and location endpoints; role checks	Review tables and food-safety check model	Safety checklist and partner reviews	Integration tests, documentation
Impact Analytics (Y. Ben Abdelkader)	Data model, conversion factors, seed data	Impact REST endpoints on test data	Consume `DeliveryCompleted`; idempotent processing	Monthly aggregates, dashboard, report, leaderboard; consume `BasketCollected`	Optional forecast (FastAPI); tests
Volunteer Driver (R. Ghileb)	Driver profile, vehicles, availability slots, overlap constraint	Available-driver search; slot booking at assignment	Driver location updates; slot release on cancellation	Driver reviews	Integration tests, documentation
Surprise Basket (M. Fadhel)	Basket catalogue CRUD	Orders, synchronous stock reservation in the Donation Offer service, unique pickup code, simulated payment	Release of unpaid reservations; expiry of baskets and pickup codes	Collection flow (code validated by the donor), `BasketCollected` event	Tests, documentation
Shared (whole team)	Config Server, Eureka, Gateway, Compose infrastructure, CI, front-end skeleton	Keycloak realm and JWT; front-end screens for donors and associations	RabbitMQ topology, event contracts, Notification service; front-end screens for drivers and citizens	Front-end dashboard and basket screens	Dockerization of the stack, end-to-end tests, documentation, demo

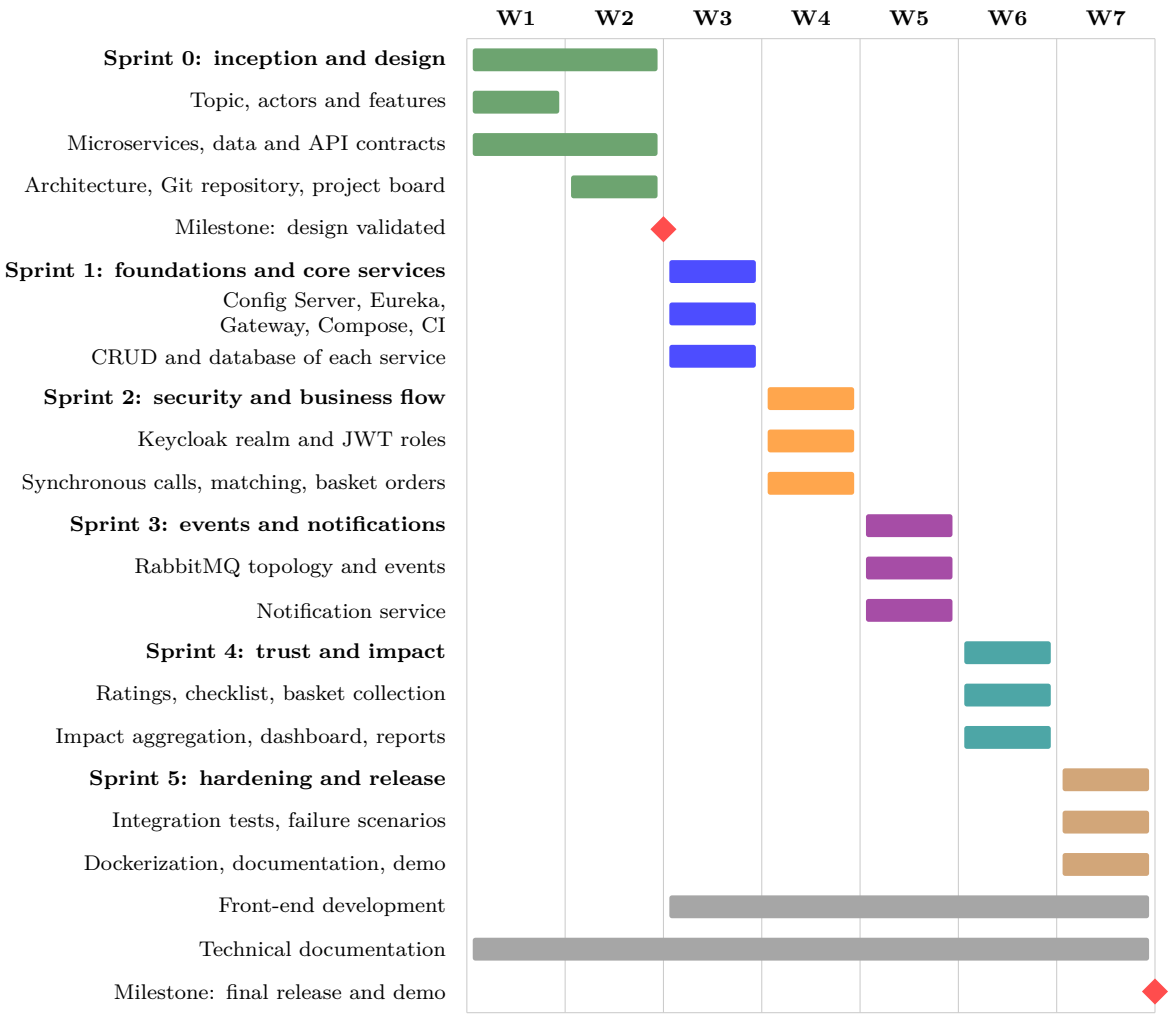


Figure 3.3: Gantt chart of the project (W1 to W7: weeks from the project start)

Chapter 4

Microservice Architecture and Service Decomposition

Domain-driven partitioning of FoodRescue TN into autonomous services, service catalogue and technology choices

This chapter explains how FoodRescue TN is decomposed into independent services. It first justifies the choice of a microservice architecture, then presents the principle used to draw the service boundaries, the catalogue of the six business services, the shared infrastructure services, the technology choices and finally the main risks of the architecture.

4.1 Why a Microservice Architecture

FoodRescue TN connects several actors (donors, associations, volunteer drivers, citizens and administrators) around business capabilities that differ strongly in nature: publishing offers, geographic matching, trust management, scheduling of drivers, statistical aggregation and e-commerce. These capabilities do not have the same data model, the same load profile or the same rate of change. A microservice architecture was therefore preferred to a monolithic application, for the following reasons.

- **Independent ownership and development:** each of the six team members owns one service end to end (code, database, tests, documentation). Services can be developed in parallel once the REST and event contracts are agreed.
- **Independent data model per service:** each service has its own database and schema, designed for its business needs: flexible documents for donation offers (MongoDB), geospatial queries for matching (PostgreSQL with PostGIS), and relational integrity and transactions for partners, drivers, impact aggregates and baskets (PostgreSQL). A change of schema in one service never affects the others.
- **Independent scalability:** the Donation Offer and Matching services receive far more traffic at peak hours than the Partner Registry. Each service can be scaled on its own.
- **Fault isolation:** a failure in one service (for example the Impact Analytics service) must not prevent donors from publishing offers or associations from collecting them.

- **Loose coupling:** services exchange facts through RabbitMQ events and never share a database, which limits the effect of a change in one service on the others.
- **Alignment with the course objectives:** the project is a distributed web application, and the decomposition exposes the real challenges of the domain: service discovery, centralised configuration, secured access, inter-service communication and eventual consistency.

The main drawbacks of this style are the additional complexity of distributed communication, the need for consistent contracts, and the difficulty of testing end-to-end flows. They are addressed by the contract-first approach and the integration tests presented in Chapter 2, by the reliability mechanisms described in Section 6.6, and by the mitigation measures of Section 4.6.

4.2 Decomposition Principle

The decomposition follows the **Domain-Driven Design** approach: the platform is divided into *bounded contexts*, each corresponding to a coherent business capability with its own vocabulary and its own data. The following rules were applied to define the boundaries.

1. **One business capability per service.** A service answers a single question of the domain (for example "what is being offered?" or "who should collect it?").
2. **Database per service.** Each service is the only owner of its data. No other service reads or writes its database directly: access goes through its REST API or through events.
3. **Single owner per service.** Each service is assigned to one team member, which clarifies responsibility and simplifies code review.
4. **Synchronous calls for questions and commands, events for facts.** A service calls another one through REST/OpenFeign when it needs an immediate answer (a validation, for instance). It publishes a RabbitMQ event when it announces something that has happened and other services may react to it.
5. **Cohesion over size.** A service is not split further as long as its data and rules change together. It is not merged with another service when its data lifecycle is different.

Applying these rules to the food rescue process gives the following chain of capabilities: *offer* the surplus, *trust* the partners, *match* offer and association, *deliver* with a volunteer, *measure* the impact, and *sell* the unsold stock as surprise baskets. Each element of this chain becomes one business service, as illustrated in Figure 4.1.

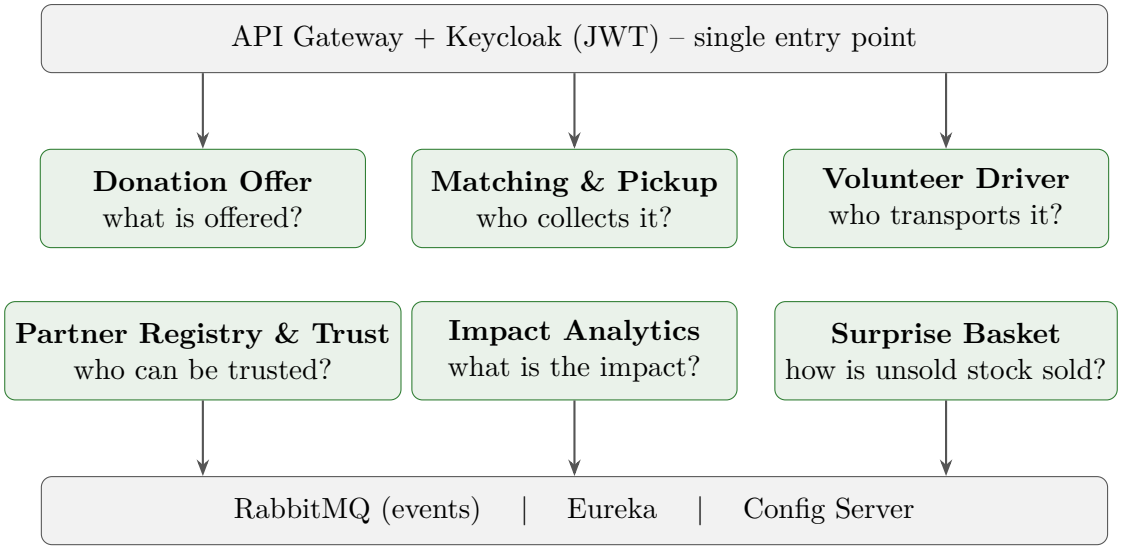


Figure 4.1: Decomposition of FoodRescue TN into bounded contexts

4.3 Service Catalogue

This section describes the six business services. Table 4.1 gives an overview, and each service is then detailed. The data models are presented in Chapter 5 and the interactions in Chapter 6.

4.3.1 Donation Offer Service

This service allows donors (bakeries, restaurants, hotels, supermarkets) to publish surplus food offers and to manage them over time.

- **Responsibilities:** create, update and cancel offers (food type, quantity, pickup window, photo, location); expire offers whose pickup window has passed; manage the stock reservations made on the offers; maintain the availability status; expose offers to authorised users.
- **Why MongoDB:** offers vary a lot from one donor to another (a tray of bread, a hotel buffet, a pallet of yoghurts). A document model stores these variable attributes and photos naturally, without a rigid schema, and supports fast filtering by category and status. An offer is stored with its food items, pickup location and photos in one document, so a stock reservation is a single atomic update.
- **Key interactions:** checks with the Partner Registry service that the donor is verified; publishes the `OfferPublished` event, which triggers the matching process; is called by the Matching service to read an offer and reserve its stock for a match; consumes `BasketOrdered` to reserve stock when part of it is sold as surprise baskets.

Table 4.1: Catalogue of the business microservices

Service	Owner	Main responsibility	Database
Donation Offer	Malek Hammami	Publication and lifecycle of surplus offers	MongoDB
Matching & Pickup	Med Sherif	Association needs, geographic matching of offers and associations, pickup scheduling, driver assignment, delivery tracking	PostgreSQL + PostGIS
Partner Registry & Trust	Salma Boubakri	Profiles of donors and associations, food-safety checklist, ratings	PostgreSQL
Impact Analytics	Yasmine Ben Abdelkader	Computation and display of kg saved, meals provided and CO_2e avoided	PostgreSQL
Volunteer Driver	Rayen Ghileb	Driver profiles, vehicles, availability slots, locations and reviews; candidate drivers for pickups	PostgreSQL
Surprise Basket Marketplace	Mohamed Fadhel	Discounted baskets, orders, pickup codes	PostgreSQL

4.3.2 Matching & Pickup Service

This service is the core of the platform. It connects each published offer with the most suitable association and organises the pickup.

- **Responsibilities:** manage the needs of associations; find the nearest compatible association (distance, accepted food categories, capacity and current need); confirm the match; schedule the pickup, assign a volunteer driver among the candidates and track the delivery status until it is completed.
- **Why PostgreSQL with PostGIS:** the matching relies on geospatial queries (distance, radius search), provided by the PostGIS extension, and on transactional consistency of matches and pickups.
- **Key interactions:** consumes `OfferPublished`; reads the offer and reserves its stock in the Offer service; reads the capacity and location of associations in the Partner Registry; asks the Volunteer Driver service for candidate drivers near the pickup point; publishes `MatchConfirmed` (towards the Notification service) and `DeliveryCompleted` (towards Impact Analytics).

4.3.3 Partner Registry & Trust Service

This service maintains the identity and reliability of the partners taking part in the food chain.

- **Responsibilities:** manage donor and association profiles and their branches (address, location, capacity) together with their verification; record the food-safety checks of branches; manage ratings and reviews.
- **Why PostgreSQL:** the data is highly structured with strong relationships (partners, checklists, ratings) and requires integrity constraints.
- **Key interactions:** answers requests from the Donation Offer service (is the donor verified?) and from the Matching service (capacity and location of associations); publishes `RatingSubmitted`, consumed by Impact Analytics to update the partner score.

4.3.4 Impact Analytics Service

This service turns operational events into measurable indicators of social and environmental impact.

- **Responsibilities:** aggregate the quantities delivered; compute kilograms of food saved, number of meals provided and CO_2e avoided, per governorate and per donor; keep the score of each partner up to date; serve the global dashboard and the leaderboard; produce a monthly report. An optional demand-forecast component, written in FastAPI, can complete these indicators.
- **Why PostgreSQL:** it stores the facts received through events and the aggregates derived from them. Pre-computed aggregate tables, indexes and grouping queries (by governorate, donor and month) keep the dashboard and the leaderboard fast without an additional cache.
- **Key interactions:** is mostly event-driven; it consumes `DeliveryCompleted`, `BasketCollected` and `RatingSubmitted`, and does not need to be called by the other services, so its availability does not affect the main flow.

4.3.5 Volunteer Driver Service

This service manages the volunteers who transport food from donors to associations.

- **Responsibilities:** register driver profiles and vehicles (type, capacity, refrigeration); manage availability slots and the last known location; keep the reviews of each driver.
- **Why PostgreSQL:** profiles, vehicles, slots and reviews are relational data where overlapping slots must be prevented by constraints.
- **Key interactions:** is queried by the Matching & Pickup service to obtain candidate drivers near the pickup point. It supplies candidates but does not schedule pickups: the assignment of a driver and the delivery status are stored by the Matching & Pickup service.

4.3.6 Surprise Basket Marketplace Service

This service offers a second outlet for unsold stock: donors can sell it at a reduced price to citizens in the form of "surprise baskets".

- **Responsibilities:** publish a catalogue of baskets built from donor stock (content description, price in TND, link to the offer); handle citizen orders; generate a pickup code; simulate the payment.
- **Why PostgreSQL:** baskets, orders, pickup codes and simulated payments are linked records with a simple lifecycle. Transactions and constraints guarantee that each order has at most one pickup code and one payment, and that each pickup code is unique. The stock itself stays in the Donation Offer service, which prevents overselling.
- **Key interactions:** publishes `BasketOrdered`, consumed by the Offer service to reserve the stock of the order (the order keeps the identifier of the reservation), and `BasketCollected`, consumed by Impact Analytics to add the kilograms saved; the Notification service can use these events to inform the citizen.

4.3.7 Choice between donation and surprise basket

For a given surplus, the donor can choose the donation route (Donation Offer and Matching services) or the sale route (Surprise Basket service). Both routes end with a measurable contribution to the impact indicators, which is why Impact Analytics listens to the events of both: `DeliveryCompleted` for donations and `BasketCollected` for sales. The stock of an offer is shared between the two routes through the reservations kept in the Donation Offer service, so that the same food is never reserved twice.

4.4 Infrastructure Services

In addition to the six business services, the platform relies on shared infrastructure components, developed and configured by the whole team. They provide the cross-cutting capabilities that the business services should not implement themselves.

These components follow standard patterns of distributed systems: *API gateway*, *service discovery*, *externalised configuration*, *centralised authentication* and *publish/subscribe messaging*. Their interplay is shown in the global architecture in Chapter 7, and the security model is detailed in Section 8.1.

4.5 Technology Choices

The technologies were selected for their maturity, their integration with the Spring ecosystem taught in the course, and their fit with the needs of each service.

Table 4.2: Infrastructure and shared services

Component	**Role in the architecture**
API Gateway	Single entry point for all client requests. Routes requests to the right service, checks the JWT token, applies cross-cutting rules (CORS, rate limiting) and hides the internal topology from clients.
Eureka (Discovery)	Service registry. Each service registers at start-up, and the gateway and Feign clients find service instances by name instead of fixed addresses, which allows scaling and relocation.
Config Server	Centralised external configuration for all services, per environment (development, test, production), so that settings can be changed without rebuilding images.
Keycloak	Identity and access management. Authenticates users and issues JWT tokens carrying the roles `DONOR`, `ASSOCIATION`, `DRIVER`, `CITIZEN` and `ADMIN`.
RabbitMQ	Message broker for asynchronous communication. Delivers domain events (such as `OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`) to the interested services without coupling the sender to the receivers.
User & Auth + Notification	Shared service for user accounts linked to Keycloak, and for notifications (e-mail or push) triggered by events, for example `MatchConfirmed` to inform the association and the driver.

Five of the six business services use PostgreSQL, as does the common User & Auth and Notification service, which reduces the learning effort since most members work with the same tools, query language and migration mechanism, and simplifies deployment: one PostgreSQL server in Docker Compose can host one separate database per service. The Donation Offer service uses MongoDB because its offers have a variable structure. Services remain independent because no database is shared: each service owns its data and exposes it only through its API and events. The price to pay is eventual consistency between services, handled through events and idempotent consumers (Section 6.6).

4.6 Main Risks and Mitigation

The decomposition into six services and shared infrastructure brings risks that were identified from the start of the project. Table 4.4 summarises them with the planned countermeasures.

4.7 Chapter Conclusion

This chapter presented the decomposition of FoodRescue TN into autonomous services. A microservice architecture was chosen because the platform combines capabilities with different data models, load profiles and rates of change, and because it allows six developers to work in parallel on well-defined boundaries.

Table 4.3: Technology choices and justification

Concern	**Technology**	**Justification**
Frontend	Angular	Responsive web application used by donors, associations, drivers, citizens and administrators; it communicates with the backend only through the API Gateway.
Service implementation	Spring Boot 3	Rapid development of REST services, large ecosystem, native integration with Spring Cloud.
Synchronous communication	REST with OpenFeign	Declarative clients, integration with Eureka for service lookup, simple contracts documented with OpenAPI.
Asynchronous communication	RabbitMQ	Reliable delivery, flexible routing with exchanges and queues, decoupling between producers and consumers.
Relational storage	PostgreSQL	Integrity constraints, transactions and a mature ecosystem (Spring Data JPA) for matches and pickups, partners, drivers, baskets and impact aggregates. One separate database per service.
Semi-structured attributes	PostgreSQL JSONB	Storage of attributes that vary between records (for example event payloads) without a second database technology.
Geospatial queries	PostGIS	Distance and radius computations for nearest-association matching.
Document storage	MongoDB	Flexible schema for donation offers, whose attributes vary from one donor to another (used only by the Donation Offer service).
Optional forecasting	FastAPI	Lightweight Python service for the optional demand forecast of Impact Analytics.
Security	Keycloak, JWT	Standard protocols (OAuth2 / OpenID Connect), role-based access, centralised user management.
Discovery and configuration	Eureka, Spring Cloud Config	Dynamic service location and centralised configuration.
Deployment	Docker, Docker Compose	Reproducible environments and one-command start of the whole system.

Table 4.4: Main risks and mitigation measures

Risk	**Impact**	**Mitigation**
Complexity of the matching logic	The Matching & Pickup service is the core of the platform and may delay the whole project.	Start with a simple nearest-first rule and improve it later (capacity, trust level, food categories).
Keycloak configuration	Blocks secured access for every service.	Prepare one shared realm with the five roles early and reuse it for all services.
Inter-service integration	Contract mismatches between services developed in parallel.	Shared Postman collection, contract-first REST and event definitions, mocked events.
Uneven workload between members	Some services are finished late and block the integration.	Weekly integration test and regular progress review.

The decomposition follows Domain-Driven Design: each bounded context became one business service, owned by one team member and holding its own database. The six resulting services cover the complete food rescue chain: *offer* (Donation Offer), *match* (Matching & Pickup), *trust* (Partner Registry & Trust), *deliver* (Volunteer Driver), *measure* (Impact Analytics) and *sell* (Surprise Basket Marketplace). They are supported by shared infrastructure components (API Gateway, Eureka, Config Server, Keycloak, RabbitMQ and the User & Auth + Notification service) that provide routing, discovery, configuration, security and messaging.

Five business services rely on PostgreSQL (with the PostGIS extension for the geographic matching), while the Donation Offer service uses MongoDB for its flexible offer documents. Spring Boot, OpenFeign and RabbitMQ are used for synchronous and asynchronous communication. The price of this independence is eventual consistency between services, which is addressed by the reliability mechanisms described in Section 6.6. The main risks of the architecture (matching complexity, Keycloak setup, integration and uneven workload) are managed through the measures of Section 4.6.

The next chapter, Chapter 5, details the data managed by each service, including its entities, attributes and ownership..

Chapter 5

Data Managed by Each Service

Class diagrams, entities, constraints and ownership of the data held by each microservice of FoodRescue TN

This chapter describes the data managed by each service of the platform. For every service it presents the class diagram of its domain model, the role of each class, the way the classes are stored, the constraints that protect the data and the references to data owned by other services. The chapter ends with a summary of data ownership.

Four design rules apply to all services, in line with the decomposition principles of Chapter 4:

- **Database per service:** each service is the only one to read and write its own database. Other services obtain its data through its REST API or through events.
- **References by identifier:** when a service needs to designate data owned by another service (a partner, an offer, a driver), it stores only the identifier of that data, for example `donorPartnerId` or `offerId`. Classes of other microservices are referenced by identifiers, never by database foreign keys, and no query joins tables of two services.
- **Snapshots for history:** data that must remain stable after an exchange is copied at the moment it is used, so that later changes in the source service do not alter past records.
- **Idempotent event handling:** services that consume events keep the identifier of the source event (`sourceEventId`) under a uniqueness constraint, so that a message delivered twice by RabbitMQ is applied only once.

All identifiers are UUIDs, including those of the MongoDB documents. Dates are date-times with time zone, and unless a unit is given in the attribute name (`quantityKg`, `priceTND`) or stored with the quantity (`unit`), quantities are expressed in kilograms.

5.1 Donation Offer Service (MongoDB)

The Donation Offer service owns the surplus offers and the stock that can be taken from them. Figure 5.1 shows its domain model. A document model was chosen because the content of an offer varies strongly from one donor to another (bread, prepared meals, fresh produce) and its parts (food items, pickup location, photos) are always read together with the offer.

4.1 Donation Offer

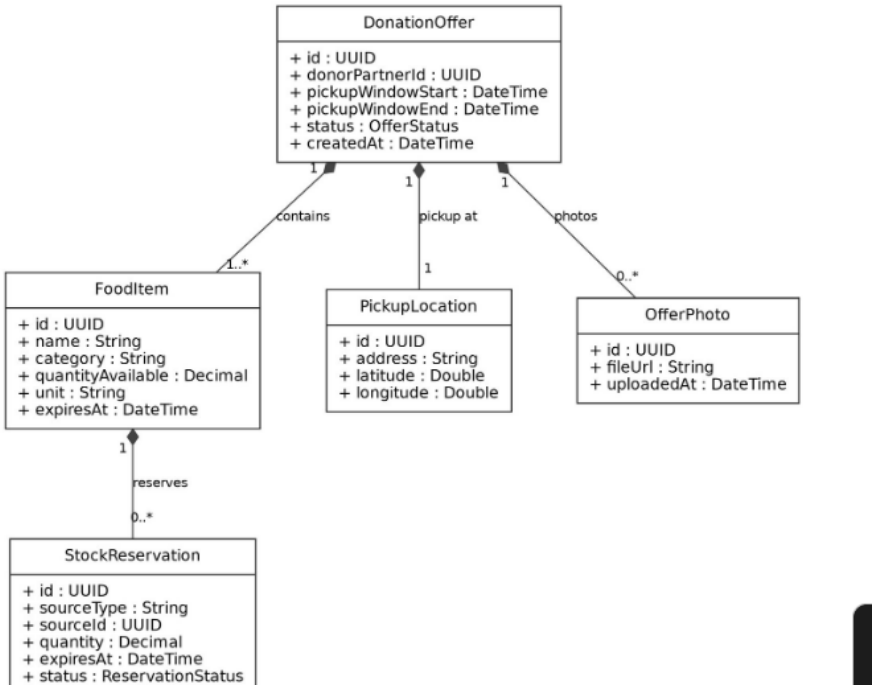


Figure 5.1: Class diagram of the Donation Offer service

Table 5.1: Classes of the Donation Offer service

Class	Storage	Role
`DonationOffer`	Collection `offers`	Root of the aggregate: donor (`donorPartnerId`), pickup window (`pickupWindowStart`, `pickupWindowEnd`), status and creation date.
`FoodItem`	Embedded array `foodItems`	One product line of the offer (name, category, unit, `quantityAvailable`, `expiresAt`). An offer contains at least one item.
`PickupLocation`	Embedded document	Address and coordinates (latitude, longitude) of the pickup point.
`OfferPhoto`	Embedded array `photos`	Address (`fileUrl`) and upload date of a photo; the image file itself is not stored in the document.
`StockReservation`	Collection `stock_reservations`	Quantity taken from a food item on behalf of a match or of a basket order: `sourceType`, `sourceId`, `quantity`, `expiresAt` and `status`.

Enumerations. `OfferStatus`: `AVAILABLE`, `RESERVED`, `COLLECTED`, `EXPIRED`, `CANCELLED`. `ReservationStatus`: `ACTIVE`, `RELEASED`, `CONSUMED`, `EXPIRED`. `sourceType`: `MATCH` or `BASKET`.

Lifecycle. An offer is created with the status `AVAILABLE`. It becomes `RESERVED` when its whole stock is reserved, and `COLLECTED` once the pickup is completed. A scheduled task sets the status `EXPIRED` when the end of the pickup window has passed and releases the reservations that were not consumed. A donor can cancel an offer that has no active reservation; cancelling a reserved offer is refused with a conflict response (HTTP 409).

Stock reservation. `FoodItem` and `PickupLocation` are embedded in the offer document, so that a reservation decreases `quantityAvailable` with one conditional update on a single document (the update applies only if the remaining quantity is sufficient), which is atomic in MongoDB. The Matching service reserves stock for a match (`sourceType` = `MATCH`), and a basket order reserves stock through the `BasketOrdered` event (`sourceType` = `BASKET`). A unique index on `(sourceType, sourceId)` prevents the same request from reserving stock twice.

Indexes. A compound index on `(status, foodItems.category)` supports the filtered listing of offers; an index on `pickupWindowEnd` supports the expiry task; an index on `(donorPartnerId, createdAt)` supports the history of a donor; an index on `(status, expiresAt)` in `stock_reservations` supports the release of expired reservations.

5.2 Matching & Pickup Service (PostgreSQL + PostGIS)

The Matching & Pickup service stores the needs of associations, the matches between offers and associations, the pickups derived from them, the drivers assigned to each pickup and the history of the pickup statuses. Figure 5.2 shows its domain model (left) together with that of the Impact Analytics service (right), which is described in Section 5.4.

Enumerations. `NeedStatus`: `OPEN`, `PARTIALLY_MATCHED`, `FULFILLED`, `CLOSED`. `MatchStatus`: `PROPOSED`, `CONFIRMED`, `REJECTED`, `CANCELLED`. `PickupStatus`: `SCHEDULED`, `ASSIGNED`, `IN_TRANSIT`, `DELIVERED`, `FAILED`, `CANCELLED`. `AssignmentStatus`: `ACTIVE`, `RELEASED`, `COMPLETED`.

Relations and constraints.

- An association need can lead to several matches (one to many), and a match schedules at most one pickup (a uniqueness constraint on the match in `pickup`).
- A pickup has many driver assignments over time, because a driver can be released and replaced, and many status events.

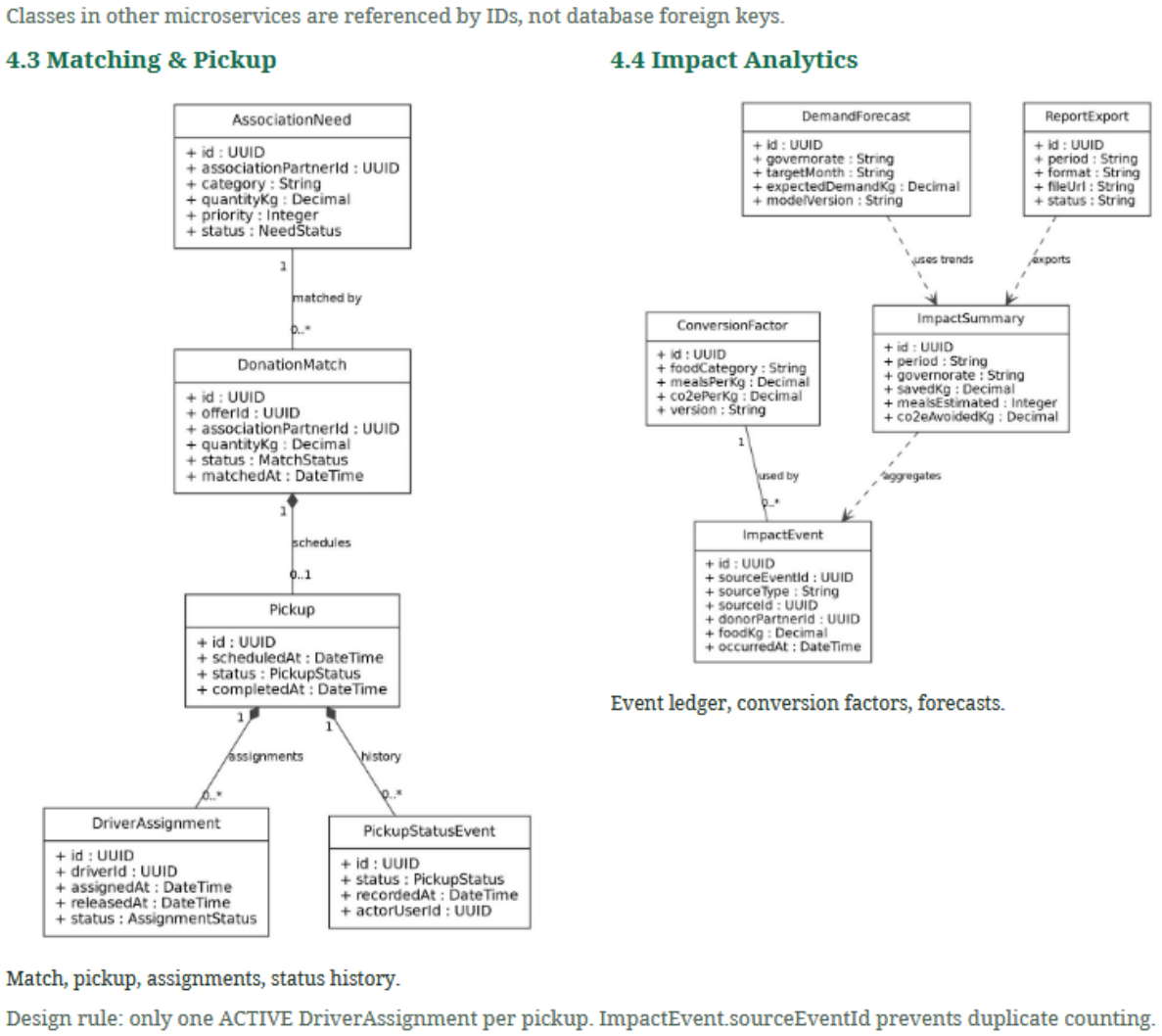


Figure 5.2: Class diagrams of the Matching & Pickup service (left) and of the Impact Analytics service (right)

Table 5.2: Tables of the Matching & Pickup service

Class	Table	Role
`AssociationNeed`	`association_need`	Need expressed by an association: `associationPartnerId`, food category, `quantityKg`, `priority` and status.
`DonationMatch`	`donation_match`	Link between one offer (`offerId`) and one association (`associationPartnerId`) for a given `quantityKg`, with its status and `matchedAt` date.
`Pickup`	`pickup`	Collection of the food, scheduled from a match: `scheduledAt`, status and `completedAt`.
`DriverAssignment`	`driver_assignment`	Assignment of a driver (`driverId`) to a pickup, with `assignedAt`, `releasedAt` and status.
`PickupStatusEvent`	`pickup_status_event`	Trace of every status change of a pickup, with the user who made it (`actorUserId`).

- **Only one active driver assignment per pickup:** a partial unique index on `driver_assignment(pickup_id)` restricted to the status `ACTIVE` prevents a pickup from receiving two drivers; a violation returns HTTP 409.
- `offerId`, `associationPartnerId`, `driverId` and `actorUserId` are references to data of other services: they are not foreign keys.
- Status columns are protected by `CHECK` constraints, and transitions are validated by the service before they are written.

Geographic data. The nearest-association rule relies on PostGIS. In the physical schema, `donation_match` additionally stores snapshots of the pickup point and of the association point as `geography(Point,4326)` columns, with the computed distance in metres; these columns are not shown in the diagram. They carry a GiST index. The query for the nearest compatible association uses a radius filter followed by an ordering by distance, which is the simple nearest-first rule adopted for the first version.

5.3 Partner Registry & Trust Service (PostgreSQL)

The Partner Registry & Trust service stores the profiles of donors and associations with their branches, the verification cases, the reviews and the food-safety checks. These data are strongly structured and linked, which suits a relational database with integrity constraints. Figure 5.3 shows its domain model.

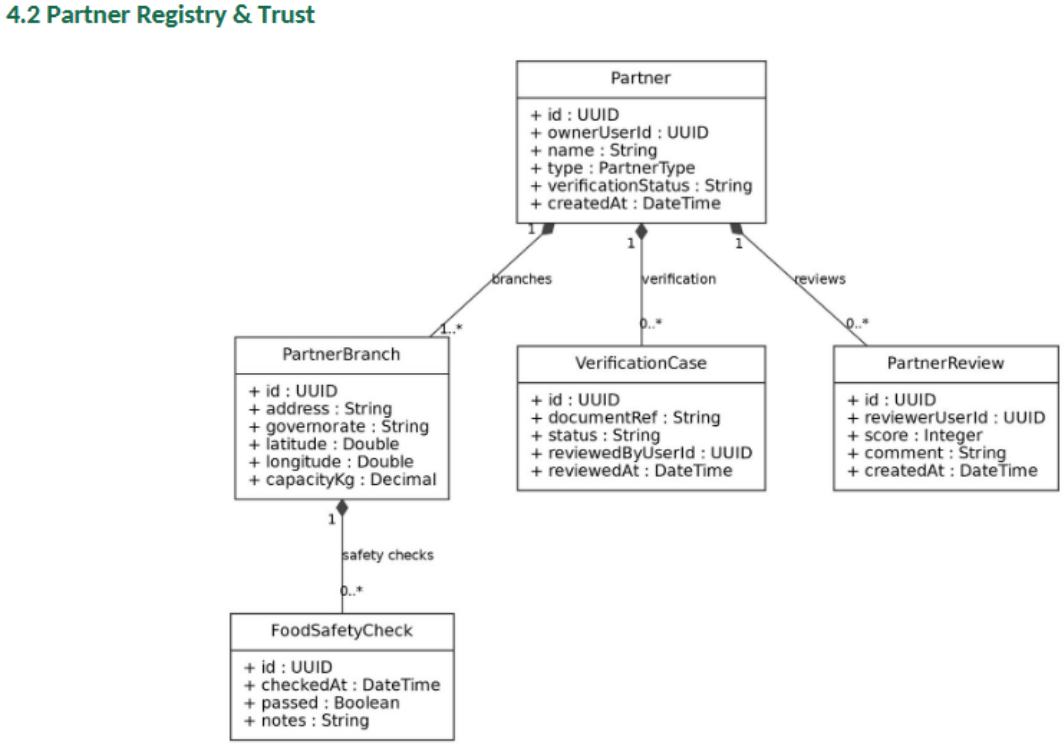


Figure 5.3: Class diagram of the Partner Registry & Trust service

Table 5.3: Tables of the Partner Registry & Trust service

Class	Table	Role
`Partner`	`partner`	Organisation taking part in the platform: `ownerUserId` (Keycloak user who manages it), name, type (`PartnerType`), `verificationStatus` and creation date.
`PartnerBranch`	`partner_branch`	Site of a partner: address, governorate, coordinates and `capacityKg`. A partner has at least one branch.
`VerificationCase`	`verification_case`	Verification file of a partner: `documentRef`, status, reviewer (`reviewedByUserId`) and review date.
`PartnerReview`	`partner_review`	Rating (`score`) and comment given to a partner by a user (`reviewerUserId`).
`FoodSafetyCheck`	`food_safety_check`	Food-safety checklist result of a branch: `checkedAt`, `passed` and `notes`.

Enumerations. `PartnerType`: `DONOR` or `ASSOCIATION`. `verificationStatus` and the status of a verification case: `PENDING`, `VERIFIED`, `REJECTED`.

Relations and constraints.

- `partner_branch`, `verification_case` and `partner_review` refer to `partner` through foreign keys; `food_safety_check` refers to `partner_branch`.
- The coordinates and the capacity are held by the branch, which is the data read by the Matching service to find an association near an offer.
- The verification of a partner is a decision of an administrator (`PUT /api/partners/{id}/verify`): it closes a verification case and updates the `verificationStatus` of the partner in the same transaction.
- `score` is limited to the range 1 to 5 by a `CHECK` constraint, and `capacityKg` must not be negative.
- `ownerUserId`, `reviewerUserId` and `reviewedByUserId` are Keycloak user identifiers, not foreign keys.

Trust. The trust level of a partner is derived from its verification status, its food-safety checks and the average of its reviews. Each new review is also published as a `RatingSubmitted` event.

5.4 Impact Analytics Service (PostgreSQL)

The Impact Analytics service stores the facts received through events and the aggregates derived from them. It owns no operational data of the other services: every row comes

from an event. Its domain model is shown on the right of Figure 5.2.

Table 5.4: Tables of the Impact Analytics service

Class	Table	Role
`ImpactEvent`	`impact_event`	Event ledger: one row per delivery or collected basket, with `sourceEventId`, `sourceType`, `sourceId`, `donorPartnerId`, `foodKg` and `occurredAt`.
`ConversionFactor`	`conversion_factor`	Parameters used to convert kilograms into meals and avoided CO_2e, per food category and `version`.
`ImpactSummary`	`impact_summary`	Pre-computed totals per `period` and governorate: `savedKg`, `mealsEstimated` and `co2eAvoidedKg`.
`DemandForecast`	`demand_forecast`	Expected demand per governorate and target month, with the `modelVersion` that produced it (optional forecasting component).
`ReportExport`	`report_export`	Export of a period report: `period`, `format`, `fileUrl` and status.

Relations and constraints.

- **No duplicate counting:** `sourceEventId` has a uniqueness constraint in `impact_event`, so that an event delivered twice is counted only once. This makes the consumption of `DeliveryCompleted` and `BasketCollected` idempotent.
- A conversion factor is used by many impact events (one to many); the factor applied to an event is the one of the version in force when the event is received.
- `impact_summary` aggregates the impact events, and `demand_forecast` and `report_export` depend on the summaries (dashed relations in the diagram). A uniqueness constraint on `(period, governorate)` gives one summary per month and governorate.
- `sourceId` and `donorPartnerId` are references to other services, not foreign keys.

Computation. When a `DeliveryCompleted` or `BasketCollected` event arrives, the service inserts a row in `impact_event` and updates the matching row of `impact_summary` in the same transaction, using the governorate carried by the event and the conversion factor of the food category. The global figures and the figures per governorate are read from `impact_summary`; the figures per donor and the leaderboard are obtained by grouping `impact_event` on `donorPartnerId`, supported by an index on `(donorPartnerId, occurredAt)`. Pre-computed summaries and indexes keep the dashboard fast without an additional cache.

5.5 Volunteer Driver Service (PostgreSQL)

The Volunteer Driver service stores the profiles of the volunteers, their vehicles, their availability slots, their last known locations and their reviews. It supplies candidate

drivers to the Matching service but does not schedule pickups: the assignment of a driver to a pickup is stored by the Matching service. Figure 5.4 shows its domain model (top) together with that of the Surprise Basket Marketplace service (bottom), which is described in Section 5.6.

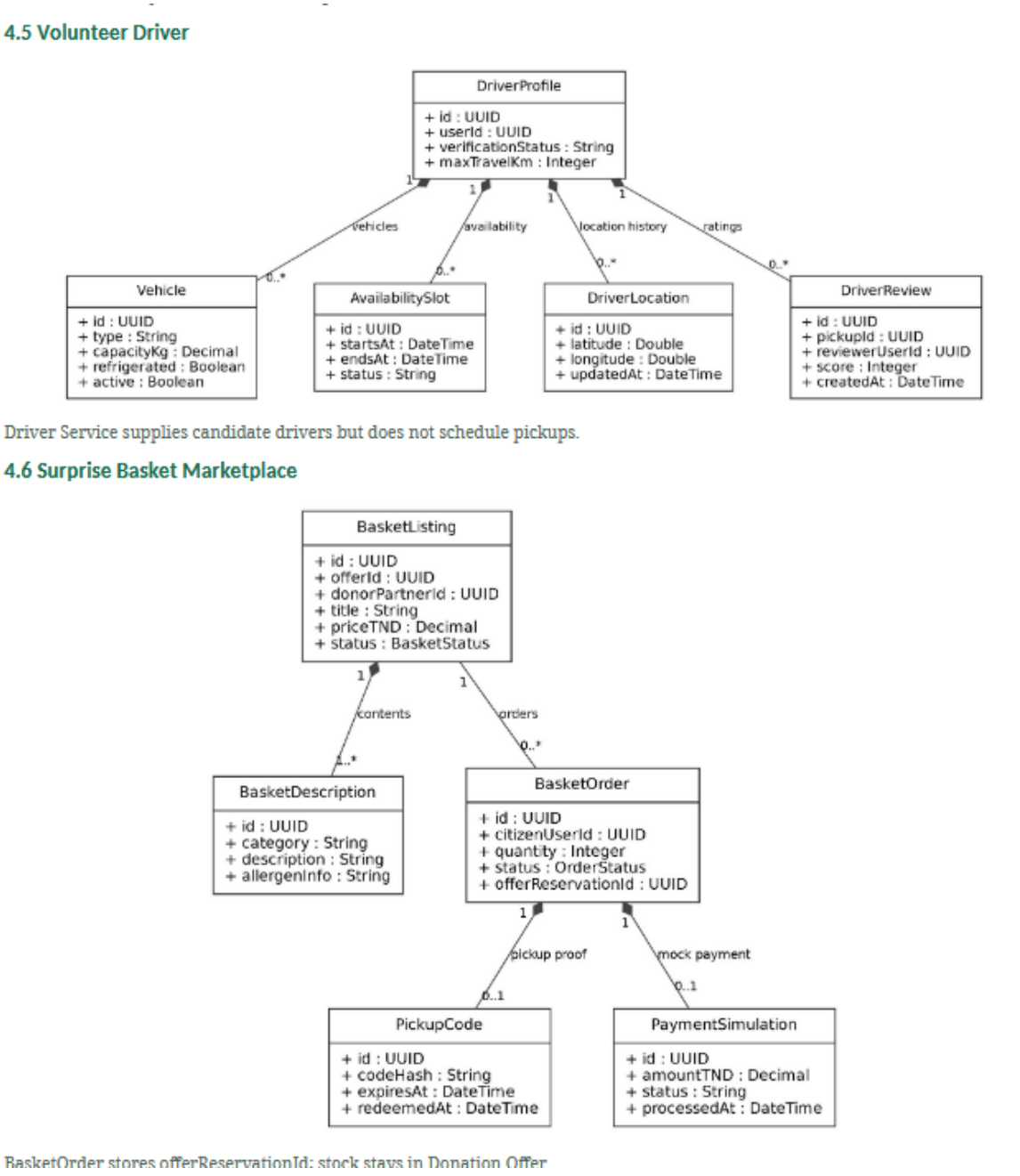


Figure 5.4: Class diagrams of the Volunteer Driver service (top) and of the Surprise Basket Marketplace service (bottom)

Relations and constraints.

- `vehicle`, `availability_slot`, `driver_location` and `driver_review` refer to `driver_profile` through foreign keys.

Table 5.5: Tables of the Volunteer Driver service

Class	Table	Role
`DriverProfile`	`driver_profile`	Profile of a volunteer: `userId` (Keycloak user), `verificationStatus` and `maxTravelKm`.
`Vehicle`	`vehicle`	Vehicle of a driver: type, `capacityKg`, `refrigerated` and `active`.
`AvailabilitySlot`	`availability_slot`	Period in which a driver can work: `startsAt`, `endsAt` and status (`FREE` or `BOOKED`).
`DriverLocation`	`driver_location`	Location history: latitude, longitude and `updatedAt`. The latest row is the current position.
`DriverReview`	`driver_review`	Review of a driver after a delivery: `pickupId`, `reviewerUserId`, `score` and date.

- An exclusion constraint on `(driver_id, time range)` prevents two overlapping availability slots for the same driver, which avoids double booking.
- `score` is limited to the range 1 to 5, and a driver can be reviewed only once per pickup and per reviewer (uniqueness constraint on `(pickup_id, reviewer_user_id)`).
- `pickupId` is a reference to the Matching service, not a foreign key.

Candidate search. The request `GET /api/drivers/available?lat=&lng=` selects the active drivers who have a free slot in the requested period, a vehicle that is active, and a last known position within `maxTravelKm` of the pickup point. The distance is computed in the query from the coordinates, with an index on `(driver_id, updatedAt)` to read the latest location.

5.6 Surprise Basket Marketplace Service (PostgreSQL)

The Surprise Basket Marketplace service stores the catalogue of baskets, the orders of citizens, the pickup codes and the simulated payments. Its domain model is shown at the bottom of Figure 5.4. The stock itself stays in the Donation Offer service: the marketplace stores only the identifier of the reservation made there.

Enumerations. `BasketStatus`: `PUBLISHED`, `SOLD_OUT`, `EXPIRED`, `CANCELLED`. `OrderStatus`: `CONFIRMED`, `COLLECTED`, `CANCELLED`, `EXPIRED`.

Relations and constraints.

- `basket_description` and `basket_order` refer to `basket_listing` through foreign keys; `pickup_code` and `payment_simulation` each belong to one order (a uniqueness constraint on the order gives at most one of each).

Table 5.6: Tables of the Surprise Basket Marketplace service

Class	Table	Role
`BasketListing`	`basket_listing`	Surprise basket put on sale: `offerId`, `donorPartnerId`, title, `priceTND` and status.
`BasketDescription`	`basket_description`	Content of a basket: category, description and `allergenInfo`. A basket has at least one description line.
`BasketOrder`	`basket_order`	Order of a citizen: `citizenUserId`, `quantity`, status and `offerReservationId`.
`PickupCode`	`pickup_code`	Proof of pickup of an order: `codeHash`, `expiresAt` and `redeemedAt`.
`PaymentSimulation`	`payment_simulation`	Simulated payment of an order: `amountTND`, status and `processedAt`.

- `pickup_code` stores only a hash of the code, which is shown to the citizen once, and the hash has a unique constraint. A code can be redeemed only once and before `expiresAt`.
- **No overselling:** the quantity is not decreased in this service. An order is confirmed only when the Donation Offer service has reserved the stock, and `offerReservationId` links the order to that reservation. The atomic decrease therefore belongs to the Offer service (Section 5.1).
- Amounts are stored in Tunisian dinars with three decimals (`NUMERIC(10,3)`), and `CHECK` constraints keep prices and quantities positive.
- `offerId`, `donorPartnerId`, `citizenUserId` and `offerReservationId` are references to other services, not foreign keys.

Events. A confirmed order publishes `BasketOrdered`, and the collection of the basket (redemption of the pickup code) publishes `BasketCollected`, which feeds the Impact Analytics service with the quantity of food collected.

5.7 User & Auth and Notification (Keycloak / PostgreSQL)

This shared component combines two kinds of data: the identity data managed by Keycloak, and the application data of the notification service stored in PostgreSQL.

5.7.1 Identity data in Keycloak

Keycloak stores the user accounts, the credentials, the roles and the registered clients in its own database. Passwords never appear in the application services: they exchange only

the JWT token issued by Keycloak, which carries the user identifier and one of the roles `DONOR`, `ASSOCIATION`, `DRIVER`, `CITIZEN` and `ADMIN`. The identifier of the Keycloak user is the common key used by the other services (`ownerUserId`, `userId`, `citizenUserId`, `reviewerUserId`, `actorUserId`).

5.7.2 Application data in PostgreSQL

Figure 5.5 shows the domain model of the application data.

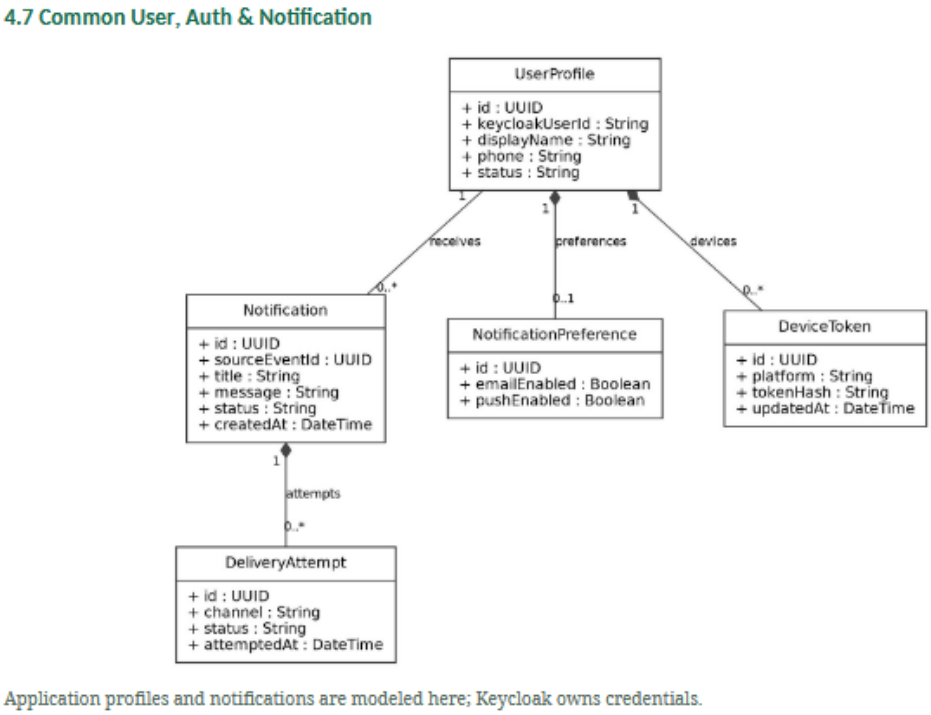


Figure 5.5: Class diagram of the common User, Auth & Notification service

Table 5.7: Tables of the User & Auth and Notification service

Class	Table	Role
`UserProfile`	`user_profile`	Application profile linked to a Keycloak account (`keycloakUserId`): display name, phone and status.
`NotificationPreference`	`notification_preference`	Channels enabled by the user: `emailEnabled` and `pushEnabled`.
`DeviceToken`	`device_token`	Device registration used for push notifications: platform, `tokenHash` and `updatedAt`.
`Notification`	`notification`	Notification triggered by an event, for example `MatchConfirmed`: `sourceEventId`, title, message, status and creation date.
`DeliveryAttempt`	`delivery_attempt`	One attempt to deliver a notification on a channel (e-mail or push), with its status and date.

Constraints.

- `keycloakUserId` is unique in `user_profile`. A user has at most one `notification_preference`, and many device tokens and notifications.
- A uniqueness constraint on `(sourceEventId, user_id)` in `notification` prevents the same event from creating the same notification twice for a user.
- A notification can have several delivery attempts, which allows a retry on the same channel or on another one after a failure. Device tokens are stored as hashes, and `tokenHash` is unique.
- Credentials remain in Keycloak: no password is stored in this service.

5.8 Data Ownership Summary

Table 5.8 summarises, for each service, the data it owns, the technology that stores it and the way other services can reach it.

Since no foreign key crosses a service boundary, the links between services are carried by identifiers. Table 5.9 lists these references and the service through which each one is resolved.

The consequence of this organisation is eventual consistency: a change made in one service becomes visible in the others only when the corresponding event has been processed. The mechanisms that handle this situation (idempotent consumers, retries) are described in Section 6.6, and the exchanges between the services are detailed in Chapter 6.

Table 5.8: Data ownership by service

Service	**Owner**	**Database**	**Data owned and access by other services**
Donation Offer	Malek Hammami	MongoDB	Offers, food items, photos and stock reservations. Read through `/api/offers`; announced by `OfferPublished`; reservations requested by the Matching service and by `BasketOrdered`.
Matching & Pickup	Med Sherif	PostgreSQL + PostGIS	Association needs, matches, pickups, driver assignments and status history. Read through `/api/pickups`; announced by `MatchConfirmed` and `DeliveryCompleted`.
Partner Registry & Trust	Salma Boubakri	PostgreSQL	Partners, branches, verification cases, reviews and food-safety checks. Read through `/api/partners`; reviews announced by `RatingSubmitted`.
Impact Analytics	Yasmine Ben Abdelkader	PostgreSQL	Impact events, conversion factors, summaries, forecasts and report exports. Read through `/api/impact`; fed only by events.
Volunteer Driver	Rayen Ghileb	PostgreSQL	Driver profiles, vehicles, availability slots, locations and reviews. Read through `/api/drivers`.
Surprise Basket Marketplace	Mohamed Fadhel	PostgreSQL	Basket listings, orders, pickup codes and simulated payments. Read through `/api/baskets` and `/api/orders`; announced by `BasketOrdered` and `BasketCollected`.
User & Auth + Notification	Whole team	Keycloak / PostgreSQL	Accounts and roles (Keycloak); profiles, preferences, device tokens and notifications (PostgreSQL).

Table 5.9: References between services (identifiers without foreign keys)

Reference	**Held by**	**Resolved through**
`donorPartnerId`	Offer, Basket, Impact	Partner Registry (`GET /api/partners/{id}`)
`associationPartnerId`	Matching	Partner Registry
`offerId`	Matching, Basket	Donation Offer (`GET /api/offers/{id}`)
`offerReservationId`	Basket	Donation Offer (stock reservation)
`driverId`	Matching (driver assignment)	Volunteer Driver
`pickupId`	Driver (review)	Matching & Pickup
`sourceId`	Offer (reservation), Impact	Matching & Pickup or Surprise Basket Marketplace
`ownerUserId`, `userId`, `citizenUserId`, `reviewerUserId`, `actorUserId`	Partner, Driver, Basket, Matching	Keycloak (JWT claim)

Chapter 6

Interactions between Services

Service dependencies, synchronous and asynchronous communication, event-driven workflows and reliability principles governing the interactions between FoodRescue TN microservices

This chapter explains how the microservices of FoodRescue TN communicate to execute the platform's main business processes. It first presents the communication principles adopted by the architecture, then shows who depends on whom through the service dependency diagram. It describes synchronous interactions through REST and OpenFeign and asynchronous interactions through RabbitMQ events, with their contracts. Two sequence diagrams illustrate the main food donation flow and the surprise basket flow. Finally, the chapter presents the reliability mechanisms required to preserve consistency and continuity in a distributed environment.

6.1 Communication Principles

FoodRescue TN is composed of six independent business microservices, each responsible for its own data and business operations, plus a common User and Notification service. Although this separation allows independent development and deployment, the services must communicate to complete processes that involve several business capabilities.

For example, publishing a food offer may trigger partner verification, geographic matching, stock reservation, driver selection, pickup scheduling, notification and impact calculation. These operations cannot be performed by a single service because each step belongs to a different bounded context.

The communication strategy follows two complementary mechanisms: **synchronous communication** and **asynchronous communication**.

- **Synchronous communication:** a service sends a request to another service and waits for its response. It is used when the requesting service needs an immediate result to continue its operation, such as checking whether a donor is verified, retrieving available drivers or reserving a quantity of food.
- **Asynchronous communication:** a service publishes an event announcing that a business action has occurred. Other services interested in that event consume it independently, without requiring the publishing service to wait for their processing.

The following rules govern communication between services:

1. **No direct database access:** a service never reads or modifies another service's database. Data is obtained through REST APIs or received through events.
2. **REST for immediate decisions:** synchronous requests are used for validations, data retrieval and operations requiring an immediate confirmation, in particular stock reservations.
3. **Events for completed business actions:** RabbitMQ events announce facts such as the publication of an offer, the confirmation of a match, the completion of a delivery or the collection of a basket.
4. **A single stock owner:** the Donation Offer service is the only service that owns and modifies the stock. The Matching and Marketplace services reserve and release quantities through its REST API, so that both routes (donation and basket sale) draw on the same stock without overselling it.
5. **Loose coupling:** event producers do not need to know the internal implementation or database structure of event consumers, and services reference each other by identifier only.
6. **Eventual consistency for events:** information carried by events (notifications, impact indicators) may take a short time to become visible in another service because event processing is asynchronous. The stock is not concerned, since it is reserved synchronously.
7. **Secure communication:** external requests pass through the API Gateway and are protected using JWT tokens issued by Keycloak. Internal service requests must also respect the authentication and authorization rules of the architecture.

Table 6.1 summarises the two communication mechanisms and their use in the platform.

Table 6.1: Comparison of communication mechanisms

Criterion	**Synchronous**	**Asynchronous**
Technology	REST / OpenFeign	RabbitMQ
Communication	Request and response	Publish and consume events
Response	Immediate response expected	No immediate consumer response
Dependency	Requester depends on the response	Producer is decoupled from consumers
Main purpose	Validation, data retrieval and stock reservation	Notification and business-event propagation
FoodRescue example	Reserving food in the Donation Offer service	Publishing `OfferPublished`

The combination of these mechanisms allows FoodRescue TN to maintain direct communication when a decision is required while distributing business events without creating unnecessary dependencies between services.

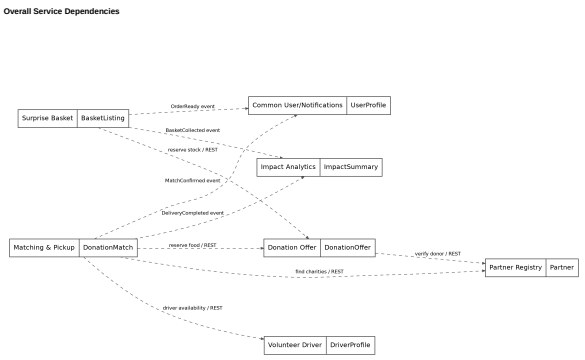


Figure 6.1: Enter Caption

6.2 Service Dependencies

Figure 6.2 gives an overview of the dependencies between the services. An arrow goes from the service that needs something to the service that provides it, and its label gives the interaction and the mode of communication (REST call or event).

Table 6.2 lists the same dependencies in detail.

Three observations can be made. First, the Donation Offer service is the most requested service: it is called by both the Matching and the Surprise Basket services because it owns the stock. Second, the Matching & Pickup service is the main coordinator of the donation workflow, as it is the only service that depends on three other services. Third, the Impact Analytics and Notification services depend on events only: no service calls them directly, so their unavailability cannot block a business operation.

6.3 Synchronous Interactions (REST / OpenFeign)

Synchronous communication is based on HTTP requests and REST APIs. Each microservice exposes endpoints that allow other services to retrieve information or execute authorised operations.

Within FoodRescue TN, OpenFeign is used as a declarative HTTP client. Instead of manually constructing HTTP requests, a service defines a Java interface containing the remote endpoints it needs to call. Spring Cloud OpenFeign generates the implementation and handles the communication.

Combined with Eureka, OpenFeign allows services to communicate using their registered service names rather than fixed IP addresses and ports. The API Gateway remains the single entry point for external client requests, while internal service-to-service calls can use service discovery directly.

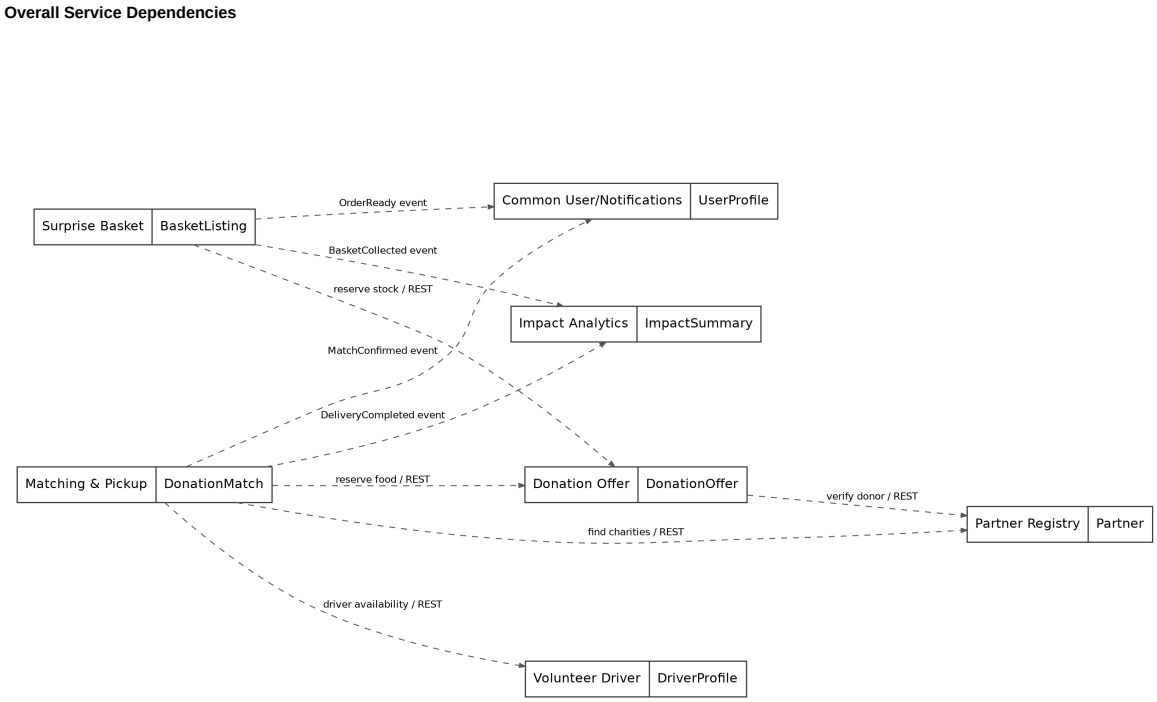


Figure 6.2: Overall service dependencies of FoodRescue TN

6.3.1 Main synchronous interactions

The synchronous interactions between FoodRescue TN services and the endpoints they use are presented in Table 6.3.

Table 6.2: Dependencies between services

From	To	Mode	Interaction
Donation Offer	Partner Registry	REST	Verify the donor
Matching & Pickup	Donation Offer	REST	Reserve food for a match
Matching & Pickup	Partner Registry	REST	Find and check charities
Matching & Pickup	Volunteer Driver	REST	Check driver availability
Surprise Basket	Donation Offer	REST	Reserve stock for an order
Donation Offer	Matching & Pickup	Event	`OfferPublished`
Matching & Pickup	Common User + Notification	Event	`MatchConfirmed`
Matching & Pickup	Impact Analytics	Event	`DeliveryCompleted`
Surprise Basket	Impact Analytics	Event	`BasketCollected`

The other endpoints of the services are called by the front-end through the API Gateway, for example publishing an offer, assigning a driver to a pickup (`POST /api/pickups/{id}/assign`), updating a pickup status (`PATCH /api/pickups/{id}/status`), placing an order or redeeming a pickup code. The complete contract of each service is given in Annex **??**.

6.3.2 Reservation of stock

The reservation is the central synchronous operation of the platform. Both the Matching service (for a donation) and the Marketplace service (for a basket order) call `POST /api/offers/{id}/reservations` with the quantity they need. The Donation Offer service checks and decreases the available quantity in one atomic operation and creates a reservation identified by its source (the match or the order). It answers with `201 Created` when the quantity is reserved and with `409 Conflict` when the remaining stock is insufficient.

If the workflow is aborted (the association rejects the match, no driver is found, the payment fails or the order is cancelled), the caller releases the reservation with `POST /api/reservations/{id}/release` and the quantity becomes available again. A reservation that is not used before its expiry date is released by the Donation Offer service itself.

6.3.3 Response codes

The services expose REST endpoints with defined HTTP response codes. These codes allow the requesting service to distinguish successful operations from validation errors, missing resources and conflicts:

- **400 Bad Request:** the request contains invalid data.

- **401 / 403:** the user is not authenticated, or not authorised to perform the operation.
- **404 Not Found:** the requested resource does not exist.
- **409 Conflict:** insufficient stock, repeated order or invalid state change.
- **503 Service Unavailable:** a critical dependency is unavailable.

6.3.4 Example of OpenFeign communication

For example, the Matching & Pickup service needs to read an offer and reserve food in the Donation Offer service. Instead of accessing MongoDB directly, it uses an OpenFeign client.

A simplified Java interface is shown below:

```
@FeignClient(name = "donation-offer-service")
public interface OfferClient {

    @GetMapping("/api/offers/{id}")
    OfferDTO getOfferById(@PathVariable("id") UUID id);

    @PostMapping("/api/offers/{id}/reservations")
    ReservationDTO reserve(@PathVariable("id") UUID id,
                           @RequestBody ReservationRequest request);

    @PostMapping("/api/reservations/{id}/release")
    void release(@PathVariable("id") UUID reservationId);
}
```

The service can then call the remote endpoints through the interface:

```
OfferDTO offer = offerClient.getOfferById(offerId);
ReservationDTO reservation = offerClient.reserve(offerId, request);
```

Eureka resolves the service name `donation-offer-service` to an available instance. The Matching service receives the response as DTO objects, without knowing how the offer and its stock are stored internally. A `409` answer is turned into a business exception, which makes the matching fail cleanly when the stock is insufficient.

This approach preserves database independence and reduces the amount of communication code that developers must write.

6.4 Asynchronous Interactions (RabbitMQ Events)

Asynchronous communication is implemented through RabbitMQ, which acts as a message broker between the microservices. It allows one service to publish a business event while other services consume that event independently.

Unlike a synchronous request, the producer does not wait for every consumer to finish processing the event. The message broker receives the published message and routes it to the appropriate queues.

This mechanism is particularly useful in FoodRescue TN because several operations are triggered by a single business action. For example, when a food offer is published, the Matching service can start the matching process without requiring the Donation Offer service to execute the entire process itself.

6.4.1 RabbitMQ communication model

The asynchronous communication model relies on the following components:

- **Producer:** the microservice that publishes an event after a business action occurs.
- **Exchange:** receives published messages and routes them to queues according to the configured routing rules.
- **Queue:** temporarily stores messages until the corresponding consumer processes them.
- **Consumer:** the microservice that receives and processes messages from a queue.
- **Binding:** connects an exchange to a queue using routing rules.

For example, the Donation Offer service acts as the producer of `OfferPublished`, while the Matching & Pickup service acts as its consumer.

The following figure illustrates the general message flow.

6.4.2 Domain events in FoodRescue TN

A domain event represents a business fact that has already happened. It is not a direct command asking another service to execute an operation.

The events exchanged between the microservices are summarised in Table 6.4.

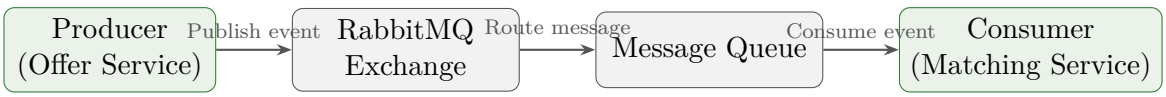


Figure 6.3: General asynchronous communication through RabbitMQ

Each event carries the information required by its consumers. For example, `DeliveryCompleted` provides the identifiers and the quantity delivered that Impact Analytics needs to calculate the corresponding impact indicators.

Table 6.3: REST endpoints used in inter-service communication

Caller → target	**Endpoint**	**Purpose**	**Codes**
Offer → Partner	GET `/api/partners/{id}`	Check that the donor is verified before accepting an offer.	200, 404
Matching → Offer	GET `/api/offers/{id}`	Read the offer and its available stock.	200, 404
Matching → Offer	POST `/api/offers/{id}/reservations`	Reserve the food quantity proposed to an association.	201, 400, 409
Matching → Offer	POST `/api/reservations/{id}/release`	Release the reservation when the match is rejected or aborted.	200, 404, 409
Matching → Partner	GET `/api/partners/{id}`	Read the verification status, branch location and capacity of an association.	200, 404
Matching → Driver	GET `/api/drivers/available?lat=&lng=`	Find available drivers near the pickup point.	200, 400
Marketplace → Offer	POST `/api/offers/{id}/reservations`	Reserve the stock of a basket order.	201, 400, 409
Marketplace → Offer	POST `/api/reservations/{id}/release`	Release the stock when the payment fails or the order is cancelled.	200, 404, 409

Every event follows a common structure made of an event identifier (`eventId`), an event type, an occurrence timestamp (`occurredAt`) and the business payload of Table 6.4. The event identifier is especially important because it allows consumers to recognise messages that have already been processed.

The stock does not travel through events. Orders of baskets and matches reserve their quantity with a synchronous call to the Donation Offer service (Section 6.3), so no event is needed to keep the quantities consistent.

6.4.3 Advantages of asynchronous communication

RabbitMQ provides several benefits to the architecture:

- **Loose coupling:** producers do not need to know the internal implementation of consumers.
- **Independent processing:** consumers process events according to their own availability and workload.
- **Improved fault isolation:** a temporary failure in Impact Analytics does not need to block the completion of a donation pickup.
- **Scalability:** consumers can be scaled independently when the number of incoming messages increases.
- **Event-driven workflows:** one business action can trigger several operations without creating a long chain of synchronous requests.

However, asynchronous communication introduces eventual consistency. For example, the global impact dashboard may not immediately reflect a delivery because the corresponding event must first be consumed and processed by Impact Analytics.

6.5 Sequence Diagrams

Sequence diagrams illustrate the chronological order of messages exchanged between actors and microservices. They make the communication mechanisms presented in the previous sections easier to understand by showing which service initiates an operation, which service responds and when events are published.

Two important workflows are presented: the donation process, from offer publication to impact calculation, and the surprise basket process, from publication to collection.

6.5.1 Main flow: from offer to impact

The main workflow begins when a donor publishes surplus food. The Donation Offer service first checks the donor's verification status through the Partner Registry service. Once the offer is accepted and stored, it publishes an `OfferPublished` event through RabbitMQ.

The Matching & Pickup service consumes this event and starts the matching process. It reads the association data from the Partner Registry (verification status, branch location and capacity) and reserves the food quantity in the Donation Offer service. It then creates a proposed match, which the association accepts or rejects. If the association rejects it, the reservation is released.

After the association accepts the match, the Matching service searches for an available volunteer driver through the Driver service, assigns the driver and schedules the pickup. It then publishes `MatchConfirmed` through RabbitMQ. The Notification service consumes this event and sends notifications to the relevant participants.

When the driver completes the delivery, the Matching service publishes `DeliveryCompleted` through RabbitMQ. Impact Analytics consumes this event, records the delivered quantity and updates the indicators for kilograms saved, meals provided and avoided CO_2e.

The sequence is illustrated in Figure 6.4.

Communication interpretation:

- Solid arrows represent synchronous REST/OpenFeign calls.
- Dashed green arrows represent asynchronous RabbitMQ events.
- RabbitMQ acts as an intermediary between event producers and consumers.
- The Matching service coordinates the operational process. It reserves the stock synchronously, so that the quantity cannot be sold as a basket while it is proposed to an association.

Table 6.4: Event contracts of FoodRescue TN

Event	Producer	Key payload	Consumer and purpose
`OfferPublished`	Donation Offer	`eventId`, `offerId`, `donorPartnerId`	Matching & Pickup: starts the matching process for a new food offer.
`MatchConfirmed`	Matching & Pickup	`eventId`, `matchId`, `associationId`, `driverId`	Common User + Notification: notifies the association and the assigned driver.
`DeliveryCompleted`	Matching & Pickup	`eventId`, `pickupId`, `foodKg`, `governorate`	Impact Analytics: records a completed donation delivery for impact calculation.
`BasketCollected`	Surprise Basket Marketplace	`eventId`, `orderId`, `foodKg`	Impact Analytics: records a collected basket for impact calculation.

- Impact Analytics and Notification process the events independently of the operational workflow.

The separation between the operational workflow and impact calculation is important. The delivery can be completed even if the impact dashboard is temporarily unavailable. The event can be processed later when the consumer becomes available.

6.5.2 Surprise basket flow

The surprise basket workflow provides an alternative to food donation by allowing citizens to purchase discounted baskets prepared from the unsold stock of a donor.

The process starts when the donor publishes a basket built from one of its offers. A citizen then browses the available baskets of a governorate and places an order. The Marketplace service asks the Donation Offer service to reserve the corresponding quantity through a synchronous call. If the stock is sufficient, the reservation is created. Otherwise the Donation Offer service answers with `409 Conflict` and the order is refused. Payment is simulated in the current project scope, and a unique pickup code is generated for the confirmed order. If the payment fails, the Marketplace releases the reservation.

Because the reservation is made in the Donation Offer service, the same stock cannot be allocated at the same time to a donation and to a basket sale. No event is needed to update the quantity.

When the citizen comes to the donor, the citizen presents the pickup code and the donor validates it (`POST /api/orders/{id}/collect`). The Marketplace updates the order status to `COLLECTED` and publishes a `BasketCollected` event through RabbitMQ.

Finally, Impact Analytics consumes the event and adds the collected basket's weight to the impact indicators.

The complete interaction is shown in Figure 6.5.

Communication interpretation:

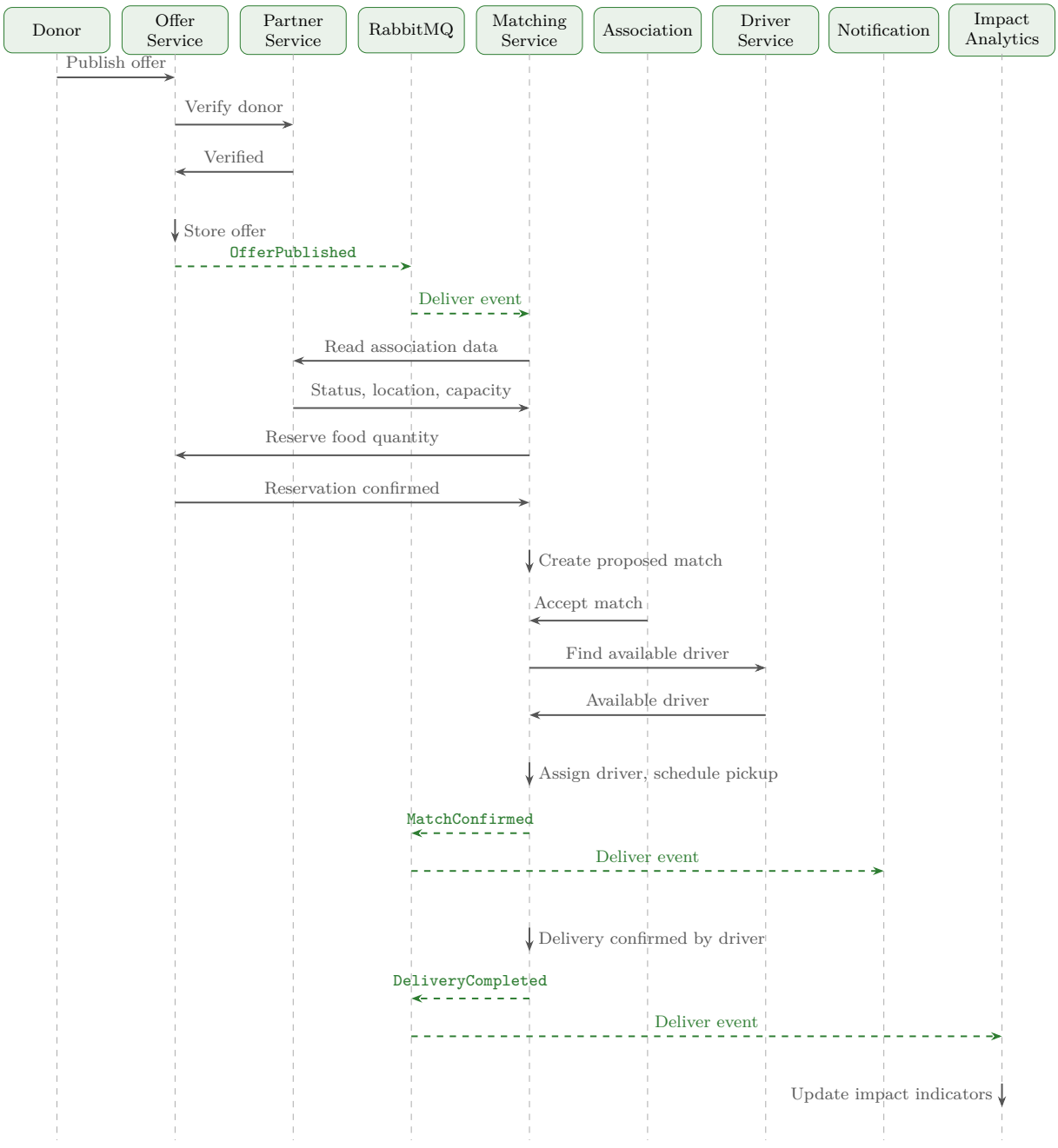


Figure 6.4: Main sequence: from offer publication to impact calculation

- The citizen and the donor communicate synchronously with the Marketplace to browse, order, publish and collect baskets.
- The Marketplace calls the Donation Offer service synchronously to reserve the stock, and releases the reservation if the order cannot be completed.
- The Marketplace publishes `BasketCollected` only after the donor has validated the pickup code.
- RabbitMQ delivers the collection event to Impact Analytics, which updates the corresponding impact indicators.

The Donation Offer service remains the single owner of the stock. The Marketplace never decreases a quantity itself: it only holds the reservation identifier of its order. This preserves database ownership and avoids the risk of overselling when a donation and a basket sale compete for the same stock.

6.6 Reliability Aspects

In a distributed architecture, communication failures can occur even when each microservice works correctly. A service may become temporarily unavailable, a request may exceed its timeout, or a RabbitMQ message may be delivered more than once.

FoodRescue TN must therefore implement reliability mechanisms that protect the business data and allow the main workflows to continue despite temporary failures.

The reliability strategy focuses on the following aspects.

6.6.1 Timeouts and error handling

A synchronous request depends on the availability of the target service. If the target service does not respond within an acceptable period, the requesting service should stop waiting and handle the failure. A Resilience4j circuit breaker opens when the failure rate of a target exceeds a threshold: the following calls fail immediately with a `503` error instead of waiting for a timeout.

For example, if the Matching service cannot contact the Partner Registry service, it must not assume that an association is verified or compatible. The matching operation is deferred until the required information becomes available.

The HTTP response codes of Section 6.3 help distinguish the situations: `409` for insufficient stock, repeated order or invalid state change, and `503` when a critical dependency is unavailable.

Timeouts prevent services from waiting indefinitely. For temporary failures, controlled retries may be used, provided that repeating the operation cannot create duplicate business actions. A retried reservation request carries the same source (match or order), so the Donation Offer service recognises it and does not reserve the stock twice.

Table 6.5 gives the behaviour of each caller when the service it calls is unavailable.

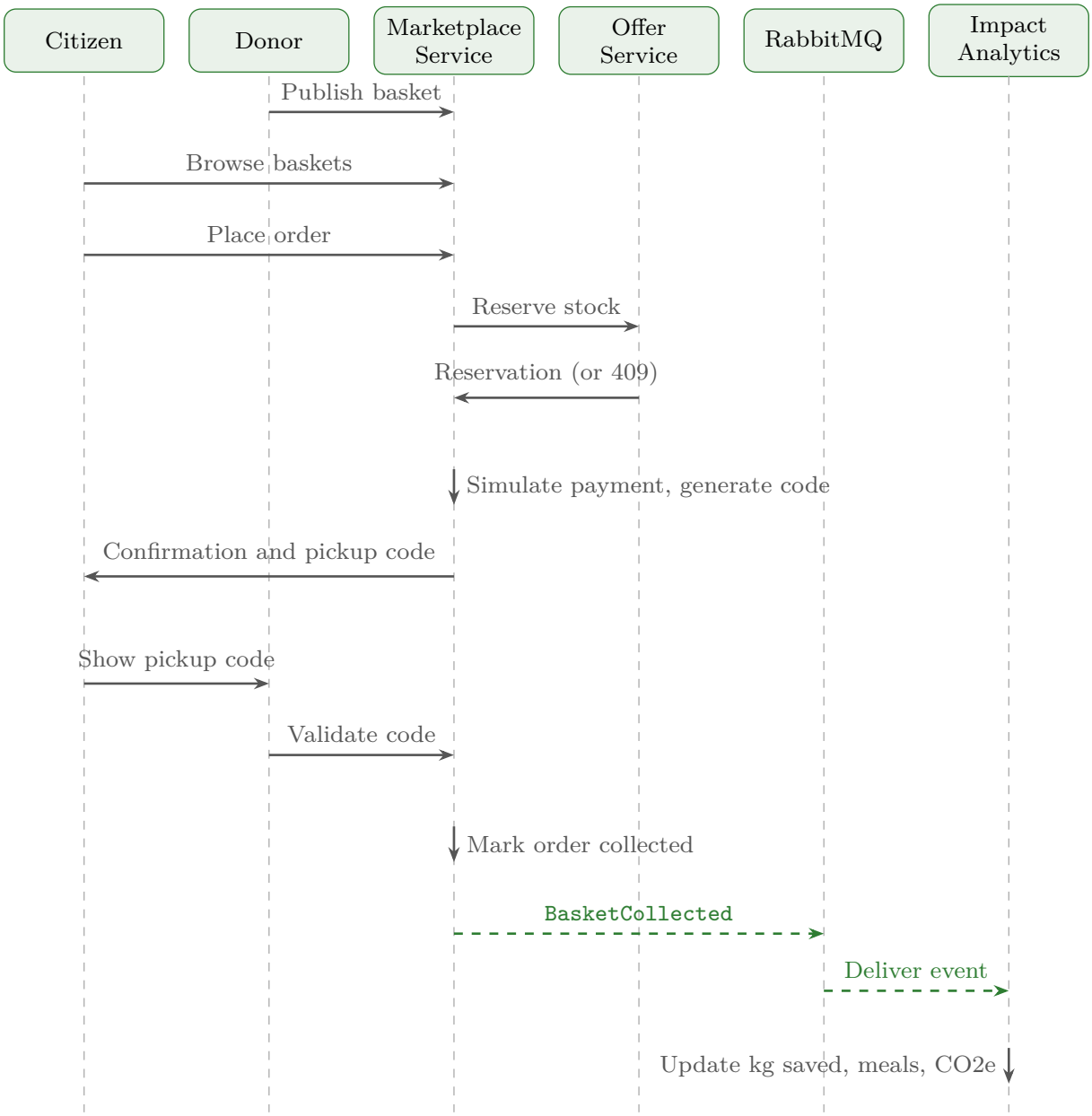


Figure 6.5: Surprise basket sequence: from order to impact calculation

6.6.2 Message delivery and retries

RabbitMQ separates message production from message consumption. However, a consumer may be temporarily unavailable or fail while processing a message.

To handle these situations, consumers should acknowledge a message only after its processing has completed successfully. If processing fails, the message can be retried according to a controlled retry policy.

A message that repeatedly fails can be redirected to a **Dead Letter Queue (DLQ)** for later inspection and reprocessing. This prevents a problematic message from blocking the normal processing of other messages.

The proposed handling strategy is summarised below:

1. The producer publishes a business event to RabbitMQ.
2. RabbitMQ routes the message to the consumer queue.
3. The consumer receives and processes the message.
4. If processing succeeds, the consumer acknowledges it.
5. If a temporary failure occurs, the message is retried.
6. If the maximum number of retries is reached, the message is sent to the Dead Letter Queue for investigation.

6.6.3 Idempotent event consumption

RabbitMQ may deliver a message more than once, for example when a consumer completes its database operation but fails before acknowledging the message.

Without protection, processing the same event twice could increase the impact indicators twice, send the same notification twice or create two matches for the same offer.

FoodRescue TN therefore adopts **idempotent event consumption**. This means that processing the same event more than once must produce the same final result as processing it once. Every event carries a unique `eventId`, and every consumer must ignore the duplicate values.

The Impact Analytics service stores the identifier of each event it has handled (the `sourceEventId` of its impact records) under a unique constraint. Before processing a new event, the service checks whether its identifier already exists. If it does, the event is ignored. Otherwise, the service records the impact fact and updates the corresponding aggregates within the same database transaction.

The same principle applies to the other consumers: the Notification service keeps the `sourceEventId` of each notification, and the Matching service ignores an `OfferPublished` event whose offer already has an active match.

6.6.4 Data consistency and concurrent operations

Since every microservice owns its own database, a single business workflow cannot rely on one transaction covering all services. FoodRescue TN uses two complementary approaches.

For the stock, which is critical, consistency is immediate: the Donation Offer service is the only owner of the stock and reserves it synchronously with an atomic conditional update. Two concurrent requests (for example a match and a basket order) cannot reserve more than the available quantity, because the second one receives a `409` answer.

For the information carried by events, such as impact indicators and notifications, consistency is **eventual**: the services become consistent after the event has been processed.

When a workflow covering several services is aborted, the reserved quantity is given back by a compensation action instead of a distributed transaction. The reservation is released when the association rejects the match, when no driver is found, when the payment fails, when the order is cancelled and when the reservation expires. A second release of the same reservation returns `409` and is treated as already done.

The following measures protect the business rules:

- **Database transactions:** each service uses local transactions to protect its own operations. For example, order confirmation and pickup-code generation are handled atomically by the Marketplace.
- **Unique constraints:** database constraints prevent duplicate active matches, duplicate pickup assignments and repeated pickup codes.
- **Conditional updates:** the available stock is checked and decreased atomically in the Donation Offer service, preventing simultaneous reservations from using more food than is available.
- **Event identifiers:** consumers store processed event identifiers to avoid applying the same business event twice.
- **Controlled retries:** temporary communication failures are handled without immediately losing business events.

6.6.5 Reliability measures by service

Table **??** summarises the main reliability measures relevant to the FoodRescue TN microservices.

These mechanisms do not eliminate every possible failure, but they reduce the risk of inconsistent data, duplicate operations and interrupted business workflows.

Table 6.5: Behaviour of the caller when a called service is unavailable

Caller	**Unavailable service**	**Behaviour of the caller**
Donation Offer	Partner Registry	The offer is refused with `503`: an unverified donor must not publish. The donor can try again later.
Matching & Pickup	Partner Registry	The matching is deferred: the `OfferPublished` message is retried and no association is considered verified.
Matching & Pickup	Donation Offer	No match is created and no stock is reserved. The message is retried and moved to the Dead Letter Queue after the last retry.
Matching & Pickup	Volunteer Driver	The accepted match stays pending without a driver and `MatchConfirmed` is not published. The search is retried, and the reservation is released if no driver is found.
Marketplace	Donation Offer	The order is refused with `503`: no payment is simulated and no pickup code is generated.
Offer, Matching, Marketplace	RabbitMQ	The event is not lost for the business operation, which is already saved. The event is published again when the broker is back (see the retry policy below).
Matching, Marketplace (producers)	Notification, Impact Analytics	No impact: events wait in the queues of the consumers and are processed when they are back. The operational workflow is not blocked.

6.7 Chapter Conclusion

This chapter presented the communication mechanisms that connect the independent microservices of FoodRescue TN. The dependency diagram showed who calls whom and by which mode. The architecture combines synchronous REST/OpenFeign interactions, used when an immediate response is required (in particular to reserve stock), with asynchronous RabbitMQ events, used to propagate completed business actions without tightly coupling the services.

The main communication flows were illustrated through two sequence diagrams. The first follows a food donation from offer publication and partner verification to stock reservation, association decision, driver assignment, delivery and impact calculation. The second describes the surprise basket process, including stock reservation, order confirmation, collection by pickup code and impact measurement.

Finally, the chapter examined the reliability challenges of distributed communication. Timeouts, circuit breakers, controlled retries, message acknowledgements, idempotent consumers and compensation of reservations help protect the system against temporary failures and duplicate operations. Because the Donation Offer service is the only owner of the stock, overselling is prevented immediately, while the information carried by events is managed through eventual consistency.

Together, these communication principles allow FoodRescue TN to coordinate its busi-

ness services while preserving service independence, data ownership and the reliability of its main food rescue workflows.

Chapter 7

Overall Architecture

> *"A well-designed architecture provides a clear organization of the system, defines the responsibilities of its components, and ensures efficient communication between services."*

This chapter presents the overall architecture of FoodRescue TN. It describes the system components, their responsibilities, and the communication mechanisms used between them. It also explains the path followed by a user request, illustrates it with the publication of a donation offer, and presents the proposed deployment organization.

FoodRescue TN follows a microservices architecture based on Angular, Spring Boot 3 and Spring Cloud. The backend is composed of six business microservices and one common User & Auth and Notification service. The services communicate through synchronous REST calls (using OpenFeign) and asynchronous events exchanged through RabbitMQ. Each microservice owns its data storage, as described in Chapter 5.

7.1 Global Microservices Architecture

The global architecture shows how the Angular frontend, the API Gateway, the business microservices, the common service, the shared infrastructure and the databases interact.

The architecture separates the public-facing client application from the backend environment. Users access the system through the Angular responsive web application. Their HTTPS requests are forwarded to the Spring Cloud API Gateway, which is the single entry point to the backend and routes requests to the seven services. The backend services run in the private network of the Docker Compose environment. Each microservice has its own business responsibility and its own database; synchronous communication uses REST and OpenFeign, while RabbitMQ carries the asynchronous business events.

Figure 7.1 presents the proposed microservices architecture of FoodRescue TN.

Interpretation of the diagram:

- **Angular frontend:** the responsive web interface used by donors, associations, volunteer drivers, citizens and administrators.
- **API Gateway:** the single backend entry point. It receives HTTP requests, validates the JWT, finds the target instance through service discovery and routes the request to the right service.

- **Six business microservices:** they manage the main domains of the platform: donation offers, matching and pickups, partner registry and trust, impact analytics, volunteer drivers and surprise baskets.
- **Common User & Auth and Notification service:** manages application-level user profiles, preferences and notifications. It is the seventh service of the diagram, but it supports the business services rather than representing one of the six business domains.
- **Keycloak:** manages identity and access. Users sign in through Keycloak, which issues OAuth2/JWT tokens carrying the roles `DONOR`, `ASSOCIATION`, `DRIVER`, `CITIZEN` and `ADMIN`.
- **Eureka:** the service registry, which maintains the list of available service instances.
- **Config Server:** centralises the configuration of the Spring Boot services.
- **RabbitMQ:** the asynchronous event bus. It carries `OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`, `RatingSubmitted`, `BasketOrdered` and `BasketCollected`, so that notification and impact processing do not block the original request.
- **Independent databases:** each microservice manages its own database. MongoDB is used by the Donation Offer service only; PostgreSQL is used by all the other services, with the PostGIS extension for the geographic operations of the Matching & Pickup service.

Note: the diagram is a logical overview of the proposed design and its main communication paths. It does not show every endpoint, event consumer or internal step. The detailed interactions are explained in Chapter 6.

7.2 Microservices and Their Responsibilities

The backend is organised into six business microservices and one common service. Each service is implemented with Spring Boot 3 and has a clearly defined responsibility.

1. Donation Offer Service (MS1)

The Donation Offer service manages the surplus offers published by donors (bakeries, restaurants, hotels, shops): creation, update, cancellation, expiry of the pickup window, food items, photos and availability status. It also manages the stock reservations made on each offer.

It uses MongoDB: an offer is stored with its food items, pickup location and photos in one document, whose flexible structure suits offers that vary from one donor to another, and a stock reservation is a single atomic update. The service calls the Partner Registry service to check that the donor is verified, and publishes the `OfferPublished` event when

an offer is published. It is called by the Matching service to read an offer and reserve its stock for a match, and it consumes `BasketOrdered` to reserve stock when part of the offer is sold as surprise baskets.

2. Matching & Pickup Service (MS2)

The Matching & Pickup service is the core of the platform. It manages the needs of associations, matches each published offer with the nearest compatible association, confirms the match, schedules the pickup, assigns a volunteer driver among the candidates and tracks the delivery status.

It uses PostgreSQL with PostGIS: PostgreSQL stores the needs, matches, pickups and driver assignments transactionally, and PostGIS provides the distance and radius computations of the nearest-first rule. The service consumes `OfferPublished`; it calls the Donation Offer service to read the offer and reserve its stock, the Partner Registry service to read the capacity and location of associations, and the Volunteer Driver service to obtain candidate drivers near the pickup point. It publishes `MatchConfirmed` and `DeliveryCompleted`.

3. Partner Registry & Trust Service (MS3)

The Partner Registry & Trust service manages the profiles of donors and associations with their branches (address, location, capacity), their verification, the food-safety checks of branches, and the ratings and reviews.

It uses PostgreSQL, which suits structured data with strong relationships. It answers the verification requests of the Donation Offer service and the capacity and location requests of the Matching service, and it publishes `RatingSubmitted` when a review is recorded.

4. Impact Analytics Service (MS4)

The Impact Analytics service turns operational events into indicators of social and environmental impact: kilograms of food saved, meals provided and CO_2e avoided, per governorate and per donor, together with a leaderboard and a monthly report. An optional demand-forecast component can be developed with FastAPI.

It uses PostgreSQL: an event ledger whose unique source-event identifier prevents duplicate counting, and pre-computed summaries that keep the dashboard fast. It is mostly event-driven: it consumes `DeliveryCompleted`, `BasketCollected` and `RatingSubmitted`, so its availability does not affect the main flow.

5. Volunteer Driver Service (MS5)

The Volunteer Driver service manages driver profiles, vehicles, availability slots, last known locations and reviews.

It uses PostgreSQL, whose constraints prevent overlapping availability slots. The Matching service queries it synchronously to obtain candidate drivers near the pickup point. The service supplies candidates but does not schedule pickups: the assignment of a driver is stored by the Matching service.

6. Surprise Basket Marketplace Service (MS6)

The Surprise Basket Marketplace service offers a second outlet for unsold stock: it manages the catalogue of discounted baskets built from donor stock, the orders of citizens, the pickup codes and the simulated payment.

It uses PostgreSQL, whose transactions and constraints keep orders, pickup codes and payments consistent. The stock itself stays in the Donation Offer service: the order only keeps the identifier of the reservation. The service publishes `BasketOrdered`, consumed by the Donation Offer service to reserve the stock, and `BasketCollected`, consumed by the Impact Analytics service to add the kilograms saved.

7. Common User & Auth and Notification Service

The common service manages application-level user profiles, notification preferences, device tokens and notifications (e-mail or push), with the delivery attempts of each notification. It is implemented with Spring Boot 3 and uses PostgreSQL.

It is distinct from Keycloak: Keycloak handles identity and authentication, whereas the common service handles application-level information and notification operations. It consumes business events such as `MatchConfirmed` to inform the association and the driver.

7.3 Explanation of the Request Path

The request path describes how a user action travels through FoodRescue TN, from the Angular frontend to the responsible microservice and, when necessary, to other services.

1. User Request and API Gateway

The process begins when a user performs an action in the Angular application, for example a donor publishing a surplus offer or a citizen browsing the available surprise baskets. The frontend sends the request over HTTPS to the Spring Cloud API Gateway.

The main responsibilities of the Gateway are:

- receiving the incoming HTTP requests of the frontend;
- providing one common entry point to the backend APIs;
- validating the JWT of protected requests;

- using service discovery to find the right service instance;
- routing the request and returning the response to the frontend.

This avoids exposing each microservice directly to external users.

2. Authentication and Authorization

Users sign in through Keycloak, which issues an OAuth2/JWT token after successful authentication. The client sends this token in the `Authorization` header (Bearer scheme) of protected requests. The Gateway validates the token before routing the request, and each microservice applies the permissions required by its own operations. For example, a donor can publish offers, while a volunteer driver can access and update the pickups assigned to them.

Keycloak therefore centralises identity and authentication, while the application services enforce the access rules of their business functions.

3. Service Discovery and Configuration

Eureka maintains a registry of the available service instances. Each service registers at start-up, which allows the Gateway and the OpenFeign clients to find instances by name instead of fixed addresses. The Spring Cloud Config Server centralises the configuration of the services per environment, which keeps the settings consistent.

4. Business Processing and Inter-Service Communication

The responsible microservice processes the request according to its business logic and accesses only its own database. When it needs an immediate answer from another service, it uses synchronous REST/OpenFeign communication. For example:

- the Offer service asks the Partner Registry service whether the donor is verified;
- the Matching service reads an offer and reserves its stock in the Offer service;
- the Matching service reads the capacity and location of associations in the Partner Registry service;
- the Matching service asks the Volunteer Driver service for candidate drivers near the pickup point.

For operations that do not require an immediate answer, services communicate asynchronously through RabbitMQ: a service publishes a business event and the interested consumers process it independently, so the original HTTP request is not kept waiting.

5. Data Management

Each microservice owns and manages its dedicated database, following the database-per-service principle. Table 7.2 lists the storage technology of each service. A service accesses its own database directly; any other service must use its API or its events and never accesses its database, and data of another service is referenced only by identifier. Keycloak manages identity data separately from the application databases.

6. Asynchronous Events, Responses and Notifications

Once the responsible service has processed the request, it returns the synchronous response through the Gateway to the frontend. Additional operations continue asynchronously through RabbitMQ. Table 7.1 lists the main business events.

Table 7.1: Main business events exchanged through RabbitMQ

Event	**Producer → Consumer**	**Purpose**
`OfferPublished`	Offer → Matching	Start the matching of a new offer.
`MatchConfirmed`	Matching → Notification	Notify the association and the driver.
`DeliveryCompleted`	Matching → Impact	Add kilograms and meals to the statistics.
`RatingSubmitted`	Partner → Impact	Update the partner score.
`BasketOrdered`	Marketplace → Offer	Reserve the stock of the order.
`BasketCollected`	Marketplace → Impact	Add the kilograms saved to the statistics.

7.4 Example: Publishing and Processing a Donation Offer

The following example shows how synchronous communication and asynchronous events work together when a donor publishes a surplus offer.

1. The donor submits a new offer (food items, pickup window, pickup location, photos) through the Angular frontend in an HTTPS request.
2. The request reaches the API Gateway, which validates the JWT and routes it to the Donation Offer service.
3. The Offer service calls the Partner Registry service to check that the donor is verified.
4. The Offer service stores the offer in its MongoDB database and returns the response (HTTP 201) through the Gateway.
5. The Offer service publishes the `OfferPublished` event to RabbitMQ.

6. The Matching service consumes the event and starts the matching process. It reads the offer in the Offer service, and reads the capacity and location of the associations in the Partner Registry service.

7. The Matching service selects the nearest compatible association with a PostGIS query and reserves the corresponding stock in the Offer service.

8. The Matching service asks the Volunteer Driver service for candidate drivers near the pickup point, assigns one of them and stores the assignment. Once the match is confirmed, it publishes `MatchConfirmed`; the Notification service informs the association and the driver.

9. When the delivery is completed, the Matching service publishes `DeliveryCompleted`, and the Impact Analytics service adds the kilograms, meals and CO_2e to the statistics.

This scenario shows the complementary roles of the two communication styles: synchronous calls are used when a service needs an immediate result (donor verification, stock reservation, candidate drivers), whereas events let notification and impact processing run independently of the original request.

7.5 Deployment View

The deployment view describes how the components of FoodRescue TN are organised in the proposed runtime environment. While the global architecture focuses on the logical structure and the communication between services, the deployment view focuses on containers, networks and infrastructure. The design uses Docker Compose to define the application containers and their networks, and distinguishes the public-facing Angular application from the private backend and infrastructure environment.

1. Public-Facing Application Layer

This layer contains the Angular responsive web application used by donors, associations, volunteer drivers, citizens and administrators. Users authenticate through Keycloak and the frontend then sends the Bearer JWT with its protected HTTPS requests to the API Gateway.

2. Backend Application Layer

The backend is composed of seven Spring Boot 3 services: Donation Offer, Matching & Pickup, Partner Registry & Trust, Impact Analytics, Volunteer Driver, Surprise Basket Marketplace, and the common User & Auth and Notification service. The API Gateway is the entry point to all of them. The services communicate through REST/OpenFeign for synchronous operations and through RabbitMQ for asynchronous events.

3. Shared Infrastructure Layer

- **Spring Cloud API Gateway:** receives the external HTTP requests, validates the JWT and routes the requests.
- **Eureka:** service registration and discovery.
- **Spring Cloud Config Server:** centralised configuration.
- **Keycloak:** identity, OAuth2 and JWT-based authentication.
- **RabbitMQ:** asynchronous event bus for business events, notifications and impact updates.
- **Docker and Docker Compose:** package the services with their dependencies and define the containers, their configuration and the networks of the environment.

4. Data Storage Layer

The data storage layer contains the databases owned by the microservices. Table 7.2 shows the technology of each service; the detailed data models are given in Chapter 5.

Table 7.2: Microservices and data storage technologies

Microservice	Technology	Main responsibility
Donation Offer (MS1)	Spring Boot 3 + MongoDB	Surplus offers, food items, stock reservations
Matching & Pickup (MS2)	Spring Boot 3 + PostgreSQL + PostGIS	Association needs, nearest-association matching, pickups, driver assignment
Partner Registry & Trust (MS3)	Spring Boot 3 + PostgreSQL	Partners and branches, verification, food-safety checks, reviews
Impact Analytics (MS4)	Spring Boot 3 + PostgreSQL	kg saved, meals, CO_2e avoided, reports
Volunteer Driver (MS5)	Spring Boot 3 + PostgreSQL	Driver profiles, vehicles, slots, locations, reviews
Surprise Basket Marketplace (MS6)	Spring Boot 3 + PostgreSQL	Baskets, orders, pickup codes, simulated payments
User & Auth + Notification	Spring Boot 3 + Keycloak + PostgreSQL	Profiles, preferences, notifications

5. Deployment Organization

Docker packages the application services and their dependencies, and Docker Compose defines and starts the required containers and networks, with one separate database per service (one PostgreSQL server can host several databases in the development environment). The Angular frontend reaches the backend only through the API Gateway, while

the services and the shared infrastructure operate inside the private network. Eureka supports service discovery, Config Server centralises configuration, Keycloak manages identity and JWT issuance, and RabbitMQ handles the asynchronous events.

The architecture also supports independent scaling: if the number of offers grows, additional instances of the Donation Offer service can be started without scaling the other services. The exact production hosting environment can be chosen according to the final implementation and infrastructure requirements.

6. Deployment Advantages

- **Modularity:** each microservice has a clearly defined business responsibility.
- **Independent deployment:** services can be maintained and deployed separately.
- **Scalability:** individual services can be scaled according to their workload.
- **Data ownership:** each service manages its own database, which reduces direct data dependencies.
- **Flexible communication:** REST/OpenFeign supports synchronous interactions and RabbitMQ supports asynchronous event processing.
- **Centralised infrastructure:** Eureka, Config Server and Keycloak provide shared discovery, configuration and identity management.
- **Simplified local deployment:** Docker and Docker Compose start the whole system in a consistent environment.

Conclusion

This chapter presented the overall microservices architecture of FoodRescue TN: the Angular frontend, the Spring Cloud API Gateway, the six business microservices and the common User & Auth and Notification service. It explained the responsibility of each service, the request path from the browser to the services, an example of offer publication and the proposed deployment environment.

The architecture separates the main business domains into independent Spring Boot 3 services. Each service owns its data: MongoDB for the Donation Offer service, PostgreSQL for the other services, and PostGIS for the geographic matching. Keycloak provides identity and JWT-based authentication, while Eureka and Spring Cloud Config provide service discovery and centralised configuration.

Synchronous REST/OpenFeign communication lets services exchange information when an immediate answer is required, and RabbitMQ complements it with the events `OfferPublished`, `MatchConfirmed`, `DeliveryCompleted`, `RatingSubmitted`, `BasketOrdered` and `BasketCollected`. Finally, Docker and Docker Compose provide the container-based deployment. Together,

these choices give FoodRescue TN a modular and maintainable foundation, from the management of surplus offers and the organisation of pickups to the sale of surprise baskets and the measurement of social and environmental impact.

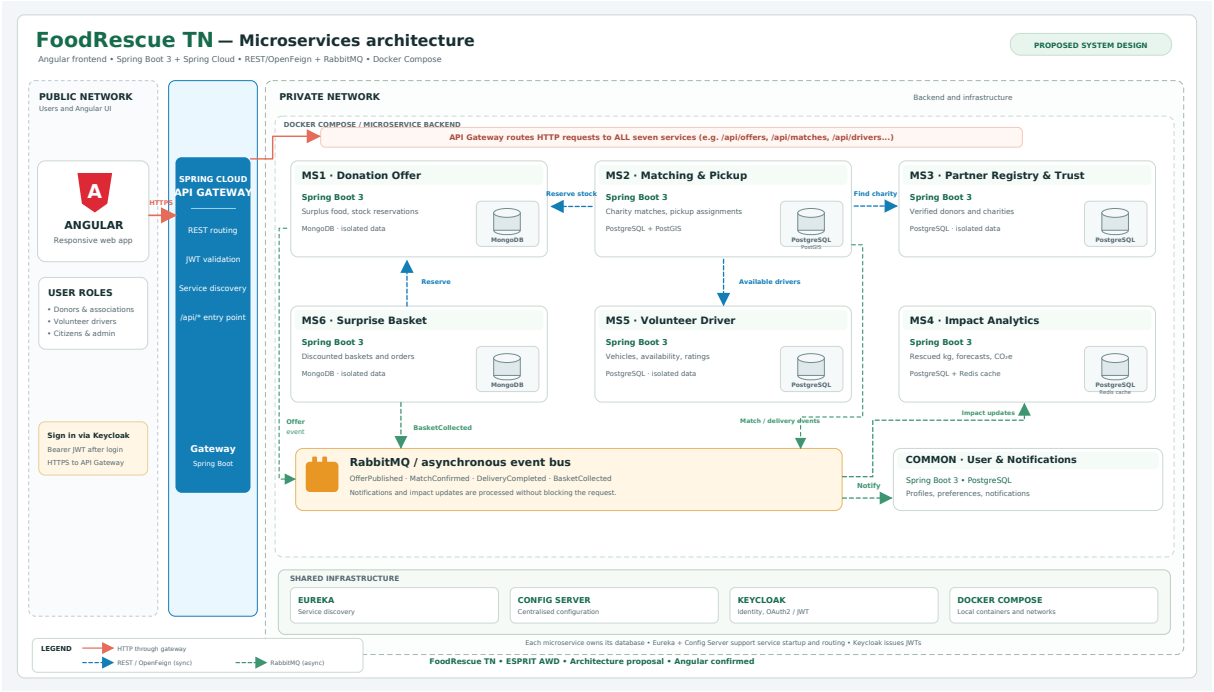


Figure 7.1: Overall microservices architecture of FoodRescue TN

Chapter 8

Other Design Elements

This chapter presents the additional design elements required for the development and operation of FoodRescue TN. It focuses on the security mechanisms, business rules, development team organization, and technology stack adopted for the proposed system.

These elements complement the microservices architecture and service interactions presented in the previous chapters. They provide a common framework for implementing the platform consistently, securely, and reliably.

8.1 Security (Keycloak, JWT, Roles)

Security is an essential aspect of FoodRescue TN because the platform handles user accounts, partner information, donation offers, pickup operations, and marketplace orders.

The proposed system uses centralized authentication through Keycloak, JSON Web Tokens (JWT), and Role-Based Access Control (RBAC) to protect its resources and restrict access to authorized operations.

8.1.1 Authentication with Keycloak

Authentication is the process of verifying the identity of a user before allowing access to protected functionalities.

FoodRescue TN uses Keycloak as its centralized identity and access management solution. Keycloak is responsible for authenticating users and issuing access tokens after successful authentication.

The general authentication process is as follows:

1. The user attempts to access the application.
2. The application redirects the user to Keycloak for authentication.
3. The user provides their credentials.
4. Keycloak verifies the credentials.
5. After successful authentication, Keycloak issues a JWT access token.
6. The client includes this token in subsequent requests to protected resources.

This approach centralizes identity management and avoids implementing a separate authentication mechanism in every business microservice.

8.1.2 JSON Web Token (JWT)

A JSON Web Token (JWT) is a compact token used to securely transmit information about an authenticated user between application components.

After successful authentication, Keycloak generates a JWT that the client includes in the HTTP Authorization header when accessing protected APIs.

The general format is:

```
Authorization: Bearer <JWT_TOKEN>
```

The token contains claims such as:

- **Subject (sub):** identifies the authenticated user.
- **Issuer (iss):** identifies the token issuer, such as Keycloak.
- **Expiration (exp):** indicates when the token becomes invalid.
- **Roles:** provide information about the permissions assigned to the user.

The API Gateway validates incoming tokens before forwarding requests to protected resources. Protected microservices also enforce the necessary authorization checks.

JWT allows the different services to verify the user's identity without requiring the user to authenticate again for every request.

8.1.3 Role-Based Access Control (RBAC)

FoodRescue TN applies Role-Based Access Control (RBAC). This means that permissions are assigned according to the role of each user.

Each user receives an appropriate role that determines which functionalities they can access and which operations they are allowed to perform.

The main roles are:

- **DONOR:** represents bakeries, restaurants, hotels, and shops that publish surplus food offers and manage their donations.
- **ASSOCIATION:** represents charities and associations that receive suitable donations, organize their collection, and provide feedback.
- **DRIVER:** represents volunteer drivers who manage their availability and handle assigned pickups.
- **CITIZEN:** represents individuals who browse available surprise baskets and place orders.
- **ADMIN:** represents system administrators who manage users, verify partners, monitor platform activities, and supervise the overall operation.

Table 8.1 summarizes the main access permissions associated with each role.

8.1.4 Authorization and Access Control

Authorization determines whether an authenticated user has permission to perform a specific action.

For example:

- A donor can modify or cancel only their own donation offers.
- A driver can update the status of pickups assigned to them.
- A citizen can access and manage only their own basket orders.
- An administrator can access authorized administrative functionalities.
- Only authorized administrators can verify partners and perform restricted administrative operations.

These permissions are enforced through security configurations and authorization checks in the backend services.

The system also verifies resource ownership. Therefore, possessing a valid token alone does not allow a user to modify another user's information or resources.

8.1.5 Security Objectives

The proposed security mechanisms aim to achieve the following objectives:

- **Authentication:** ensure that users are properly identified.
- **Authorization:** restrict access according to user roles and permissions.
- **Confidentiality:** protect user and operational information from unauthorized access.
- **Integrity:** prevent unauthorized modification of data.
- **Centralized identity management:** manage authentication through Keycloak.

8.2 Business Rules and Assumptions

Business rules define the conditions and constraints that govern how FoodRescue TN operates. They ensure that the different actors interact with the platform in a consistent and organized way.

The following rules describe the main expected behaviors of the system.

8.2.1 User and Partner Management

- Every user must have an account to access protected functionalities.
- Each user is associated with an appropriate role.
- Donors and associations must provide the required information before using their partner functionalities.
- Partner verification is performed by an authorized administrator.
- Users can only perform operations permitted by their roles.
- Users can access or modify only the information and resources they are authorized to manage.

8.2.2 Donation Offer Management

- A donor can publish a food surplus offer by providing information such as the food category, quantity, description, pickup location, and pickup time window.
- An offer must contain all required information before it can be published.
- An offer must be available and eligible before it can participate in the matching process.
- A cancelled, expired, or already allocated offer must not be assigned again.
- During matching, an offer must be allocated in a controlled manner to prevent multiple associations from receiving the same offer.
- Donors are responsible for providing accurate information about their food offers.
- Offers must respect the applicable food safety and storage requirements.

8.2.3 Matching and Pickup Management

- The matching process identifies suitable associations according to criteria such as location, food category, capacity, and availability.
- Geographic proximity is considered when selecting a suitable association.
- A donation offer must not be assigned to multiple associations at the same time.
- A pickup can be assigned to an available volunteer driver.
- A driver must be informed about the pickup location and scheduled time.
- Only the driver assigned to a pickup, or another authorized user, can update its status.

- Pickup status must be updated as the operation progresses.
- A pickup is considered completed only after its completion has been confirmed.
- A completed pickup can contribute to the platform's impact indicators.

8.2.4 Surprise Basket Marketplace

- Donors can publish surprise baskets containing discounted surplus food.
- Each basket must include relevant information such as its description, price, quantity, and collection period.
- Citizens can place orders only for available baskets.
- The available quantity must be updated when an order is confirmed.
- The system must prevent orders from exceeding the available basket stock.
- A basket must not be ordered beyond its available quantity, even when multiple orders are submitted at the same time.
- Collection must be confirmed before an order is considered successfully collected.
- Repeated order events must not reduce the available stock more than once.

8.2.5 Ratings and Impact Calculation

- Associations can submit ratings after relevant donation operations have been completed.
- Ratings must be associated with the concerned partner and operation.
- Impact indicators must be based on recorded and completed operations.
- The quantity of food saved is calculated using the quantities confirmed through completed operations.
- The number of meals and the estimated CO_2 equivalent avoided are calculated using documented conversion factors.
- Cancelled or unsuccessful operations must not be counted as completed food-saving activities.
- The same completed operation must not be counted more than once in the impact calculations.
- Impact Analytics receives relevant completion events through RabbitMQ before updating its indicators.

8.2.6 Main Business Assumptions

The implementation of FoodRescue TN relies on several assumptions:

- Users have access to an internet connection to interact with the platform.
- Donors provide accurate information about the quantity, category, condition, and availability of surplus food.
- Associations maintain updated information about their location, capacity, and requirements.
- Volunteer drivers update their availability and pickup status.
- Geographic information is available to support proximity-based matching.
- The platform uses agreed conversion factors to estimate meals and environmental impact.
- The actual availability of food and the completion of a pickup must be confirmed by the concerned actors.

These rules and assumptions provide a common basis for implementing the business logic across the different microservices.

Specific operational thresholds, such as maximum pickup delays and food eligibility conditions, can be configured according to the platform's requirements.

8.3 Team Organization

The development of FoodRescue TN requires coordination between different development activities. Since the system is based on a microservices architecture, the work must be organized to ensure that the different services can be developed and integrated consistently.

The team organization is based on the main technical and functional responsibilities involved in the project.

8.3.1 Development Responsibilities

The main responsibilities can be organized as follows:

- **Project Coordination:** organizes development activities, defines priorities, follows project progress, and ensures that the different tasks remain consistent with the project objectives.
- **Backend Development:** implements the Spring Boot microservices, business logic, REST APIs, and communication mechanisms.

- **Frontend Development:** develops the client interface and integrates it with the backend APIs.
- **Database Management:** designs and manages the databases, data models, and relationships required by the different services.
- **Security and Integration:** configures Keycloak, JWT, API Gateway, service discovery, and communication between microservices.
- **Testing and Quality Assurance:** verifies the main functionalities, tests API endpoints, checks integration between services, and identifies errors.

These responsibilities describe the main work areas of the project. Depending on the team size, one member may be responsible for several activities, and multiple members may collaborate on the same task.

8.3.2 Work Organization

The project development is organized into several activities:

1. **Requirements Analysis:** identifying the actors, their needs, and the main functionalities of FoodRescue TN.
2. **System Design:** defining the microservices architecture, database structures, APIs, communication mechanisms, and security rules.
3. **Implementation:** developing the frontend, backend services, databases, and shared infrastructure components.
4. **Integration:** connecting the different microservices and verifying communication through REST APIs and RabbitMQ.
5. **Testing:** checking individual functionalities and the behavior of the complete system.
6. **Documentation:** preparing technical documentation describing the architecture, design decisions, and implementation.

8.3.3 Collaboration and Coordination

Because the system contains several independent microservices, coordination is important to avoid integration problems.

The team must agree on common API contracts, data formats, event structures, naming conventions, and security rules before integrating the services.

Regular communication and shared documentation help team members understand the responsibilities of each component and identify dependencies between development tasks.

This organization encourages parallel development while maintaining consistency across the complete FoodRescue TN system.

8.4 Technology Stack Summary

FoodRescue TN relies on a combination of technologies selected according to the responsibilities of each component. The technology stack supports backend development, data storage, communication, authentication, deployment, and analytics.

The following table summarizes the main technologies used in the proposed system.

8.4.1 Database Allocation

Each business microservice manages its own data according to its functional responsibilities.

The selected database allocation is summarized below:

- **Donation Offer Service:** MongoDB is used to store donation offer information.
- **Matching and Pickup Service:** PostgreSQL with PostGIS is used to manage matching operations, pickups, and geographic data.
- **Partner Registry and Trust Service:** PostgreSQL is used to store partner information, verification details, and ratings.
- **Impact Analytics Service:** PostgreSQL is used to store operational data and calculated impact indicators.
- **Volunteer Driver Service:** PostgreSQL is used to manage driver information, availability, and assignments.
- **Surprise Basket Marketplace Service:** PostgreSQL is used to manage baskets, orders, and collection information.
- **Shared User and Auth + Notification Service:** application-level user information and notification data are managed separately from the identity management responsibilities handled by Keycloak.

Each microservice is responsible for accessing and managing its own database. Other services interact with it through the defined APIs or asynchronous events rather than accessing its database directly.

8.4.2 Technology Selection

The selected technologies provide complementary capabilities:

- **Spring Boot** provides the foundation for developing independent backend microservices.
- **Spring Cloud** provides infrastructure components such as the API Gateway, Eureka, and Config Server.
- **Keycloak and JWT** provide centralized identity management and secure access to application resources.
- **RabbitMQ** supports asynchronous communication and event-driven interactions between independent services.
- **MongoDB and PostgreSQL** provide different data storage capabilities according to the needs of each service.
- **PostGIS** supports geographic operations required by the matching process.
- **Docker and Docker Compose** facilitate application containerization and the deployment of the different components.
- **Power BI** supports the visualization and analysis of the platform's impact indicators.
- **Python and FastAPI** are considered optional technologies for possible forecasting extensions and are not required for the core microservices architecture.

The shared User and Auth + Notification application service supports application-level user information and notification functionalities. Keycloak remains responsible for centralized identity management and authentication.

The proposed architecture contains six business microservices, together with shared infrastructure and application components. These shared components support the business services without being considered additional business microservices.

Table 8.1: Main user roles and permissions

Role	**Main permissions**
DONOR	Publish, update, and cancel food offers; manage surplus information.
ASSOCIATION	View suitable offers, participate in donation matching, manage received donations, and submit ratings.
DRIVER	Manage availability, view assigned pickups, and update pickup status.
CITIZEN	Browse available surprise baskets, place orders, and confirm collection.
ADMIN	Manage users, verify partners, supervise activities, and monitor system information.

Table 8.2: Technology stack of FoodRescue TN

Category	**Technology**	**Purpose**
Backend Development	Java and Spring Boot	Development of the business microservices and REST APIs.
API Gateway	Spring Cloud Gateway	Routing and management of incoming requests.
Service Discovery	Eureka	Registration and discovery of microservices.
Configuration Management	Spring Cloud Config	Centralized configuration of services.
Authentication	Keycloak and JWT	User authentication and role-based access control.
Synchronous Communication	REST and OpenFeign	Direct communication between microservices.
Asynchronous Communication	RabbitMQ	Event exchange between independent services.
Document Database	MongoDB	Storage of donation offer information in the Donation Offer Service.
Relational Database	PostgreSQL	Storage of structured business data for the relational services.
Geographic Data	PostGIS	Geographic queries and location-based matching in the Matching and Pickup Service.
Deployment	Docker and Docker Compose	Containerization and deployment of the application components.
Analytics and Reporting	Power BI	Visualization of operational, social, and environmental indicators.
Optional Forecasting	Python and FastAPI	Possible integration of forecasting functionalities in future extensions.

General Conclusion

The increasing amount of food waste represents a major economic, social, and environmental challenge. At the same time, many individuals and organizations continue to face food insecurity. Addressing this issue requires more than simply redistributing surplus food; it requires an organized, reliable, and accessible system capable of connecting food donors with the organizations and individuals who can benefit from their resources.

In this context, **FoodRescue TN** was proposed as a digital solution aimed at reducing food waste and encouraging solidarity through technology. The main objective of the project is to facilitate the collection, redistribution, and valorization of surplus food by connecting donors, associations, volunteer drivers, and citizens through a centralized digital ecosystem.

Throughout this report, we presented the different aspects of the proposed system, beginning with the analysis of the existing problem, the identification of stakeholders, and the definition of functional and non-functional requirements. We then translated these requirements into a structured system design based on a microservices architecture. This architecture is organized around six business microservices: Donation Offer, Matching and Pickup, Partner Registry and Trust, Impact Analytics, Volunteer Driver, and Surprise Basket Marketplace. Each service has a specific responsibility and manages its own data, promoting modularity, maintainability, and independent development.

The proposed architecture also integrates shared infrastructure components and communication mechanisms to ensure effective coordination between services. Spring Cloud Gateway, Eureka, and Spring Cloud Config support request routing, service discovery, and centralized configuration, while REST and OpenFeign enable synchronous communication. RabbitMQ supports asynchronous event-driven interactions, allowing important business events to be communicated between services without creating unnecessary dependencies. Furthermore, Keycloak and JSON Web Tokens provide centralized authentication and role-based access control, helping protect the platform and its resources.

Beyond its technical architecture, FoodRescue TN aims to create practical value through two complementary approaches. The donation management process facilitates the redistribution of surplus food to associations according to criteria such as location, category, capacity, and availability. In parallel, the Surprise Basket Marketplace provides citizens with the opportunity to purchase discounted surplus food, encouraging responsible consumption while offering donors an additional way to reduce food losses. The Impact Analytics Service complements these activities by providing indicators that help monitor the platform's social and environmental contribution, including quantities of food saved, estimated meals, and CO_2 equivalent avoided.

The report also established the business rules, security requirements, development responsibilities, and technology choices necessary to maintain consistency across the proposed system. Together, these design elements provide a coherent foundation for the

future implementation and integration of the platform.

Nevertheless, the proposed solution also offers opportunities for further development. Future improvements could include advanced matching algorithms, demand forecasting, intelligent recommendations, mobile application enhancements, real-time notifications, and integrations with additional food donors and charitable organizations. The platform could also evolve through the introduction of more advanced analytics and monitoring capabilities, enabling stakeholders to make better-informed decisions and evaluate the impact of their activities more effectively.

Ultimately, **FoodRescue TN is more than a food redistribution application; it represents a step toward a more connected, responsible, and sustainable food ecosystem in Tunisia.** By combining digital innovation, coordinated logistics, and social engagement, the project seeks to transform surplus food into an opportunity for solidarity and environmental responsibility. Its long-term ambition is to contribute to a future in which less food is wasted, more resources are shared, and technology becomes an effective tool for addressing real societal challenges.

Conclusion

This chapter presented the additional design elements of FoodRescue TN. It described the security mechanisms based on Keycloak, JWT, and Role-Based Access Control, as well as the business rules and assumptions governing the main platform operations.

It also introduced the organization of the development team and summarized the technologies selected for the implementation of the system, including backend development, databases, communication, deployment, and analytics.

Together with the architecture and interaction design presented in the previous chapters, these elements provide a consistent foundation for the implementation and integration of the FoodRescue TN system.